Analysis and Design of Algorithms

CS3230

Dynamic Programming Examples

- Coin changing
- Knapsack problem
- Matrix chain multiplication

Prof Lee Wee Sun

Coin Changing

Problem: Need change for n cents. Use the fewest coin possible. There are k denomination $d_1, \ldots, d_k$.

- Always picking the largest possible coin (greedy) does not always work

Counterexample to greedy:

- Available denomination: *25¢, 10¢, 1¢*.

- Make change for *30¢*
  - Greedy picks *25¢+1¢+1¢+1¢+1¢+1¢*.
  - Optimal is *10¢+10¢+10¢*.

Optimal substructure:

An optimal solution $M[j]$ for j cents can be decomposed into $M[i]+1$, where $M[i]$ is the optimal solution for some $i < j$.

Proof: (*Cut and paste*) If $M[i]$ is not optimal, we can reduce the cost by using the optimal change for i.

Recursive solution:

There are k possible subproblems to solve for value i at: $i\text{-}d_1, i\text{-}d_2, \ldots, i\text{-}d_k$. So

$$M(i) = \begin{cases} 1 + \min_{1 \le j \le k} \{M(i - d_j)\} & \\ 0 & i = 0 \\ \infty & i < 0 \end{cases}$$

Memoized Solution

```
NUM-COINS(i)
▷ Terminal cases
if i <0 then
    return ∞
if M[i] ≠ -1 then
    return M[i]
else
    ▷ Solve subproblems
    ....
return M[i]
```

Define an array $M[0,n]$. Initialize $M[0]=0$, other elements to -1, indicating unsolved.

- Solve subproblems

for $j \leftarrow 1$ to k

$currCoins$ = **NUM-COINS**(i-d_j)+1

if $M[i] > currCoins$ **then**

$M[i] = currCoins$

Overlapping subproblems:

Only $\Theta(n)$ entries for $M[i]$.

For each entry, $\Theta(k)$ computation to check k subproblems, so total runtime $\Theta(nk)$

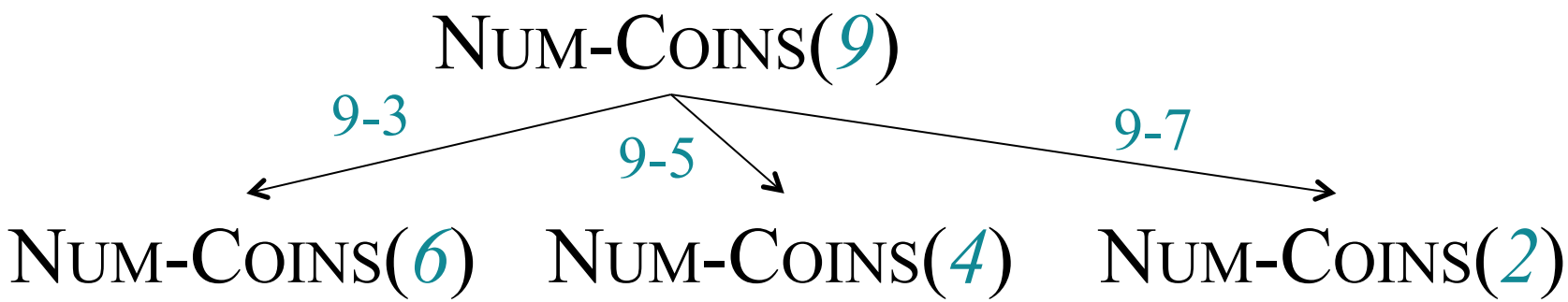
Example: $n = 9$,

denominations $3, 5, 7$

NUM-COINS(9)

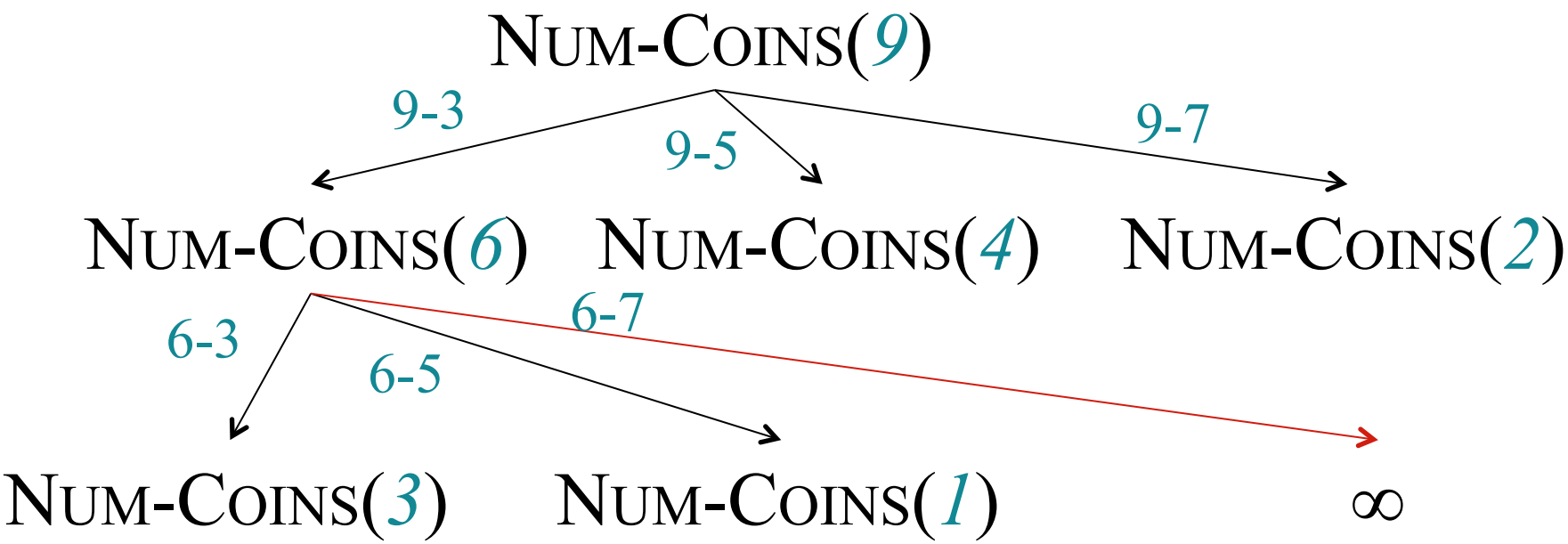
Example: $n = 9$,

denominations 3, 5, 7



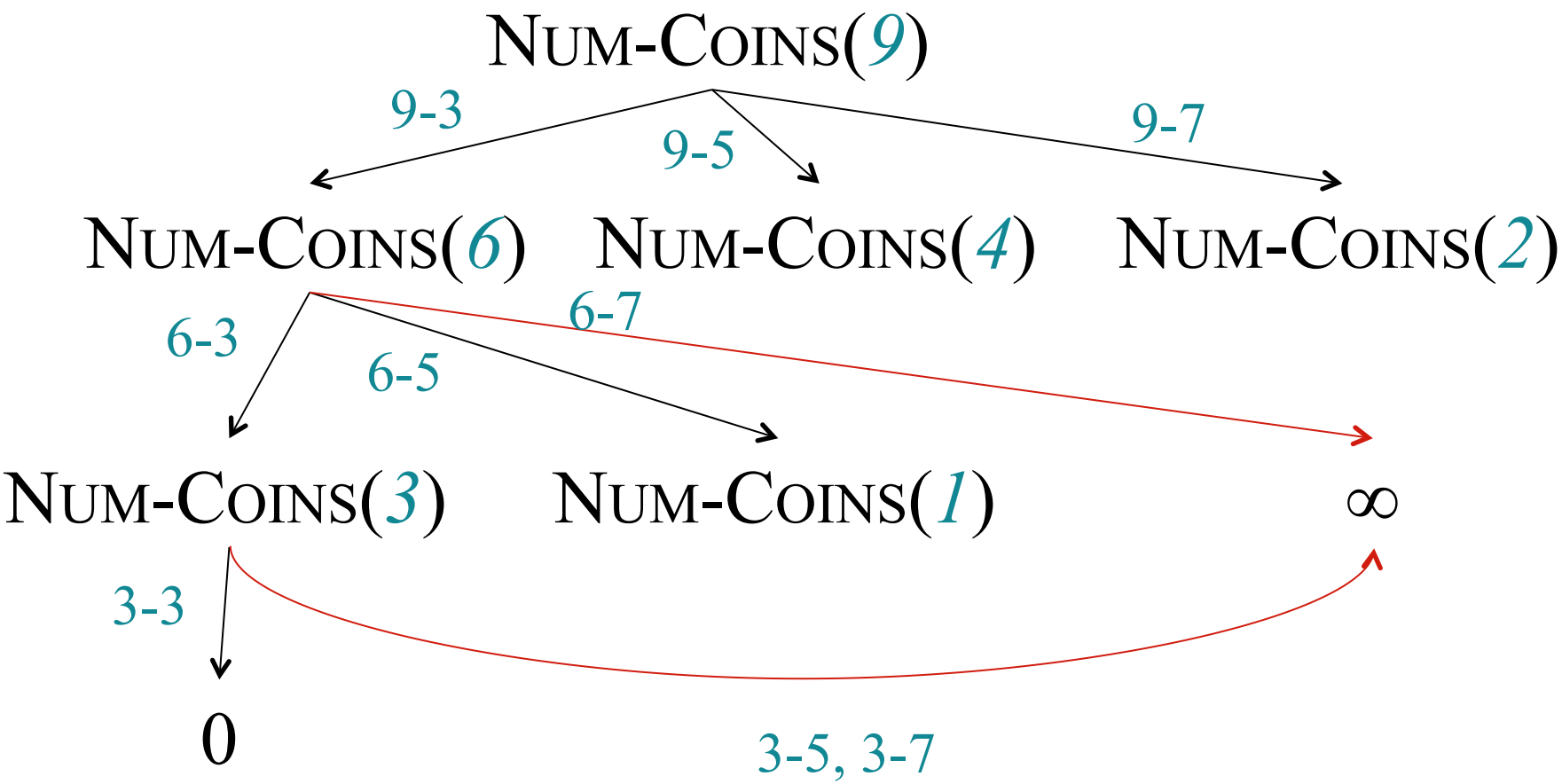
Example: $n = 9$,

denominations 3, 5, 7



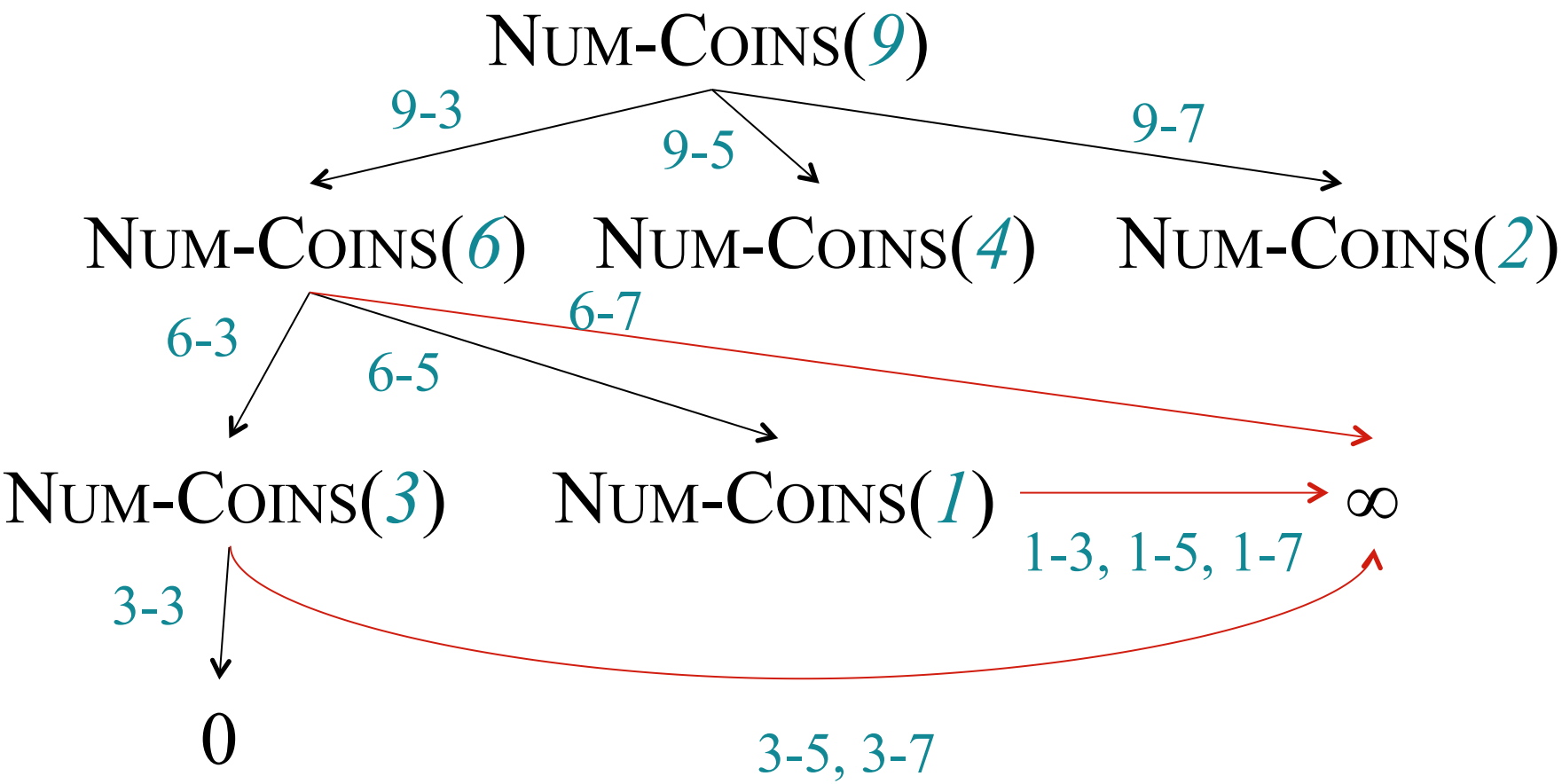
Example: $n = 9$,

denominations 3, 5, 7



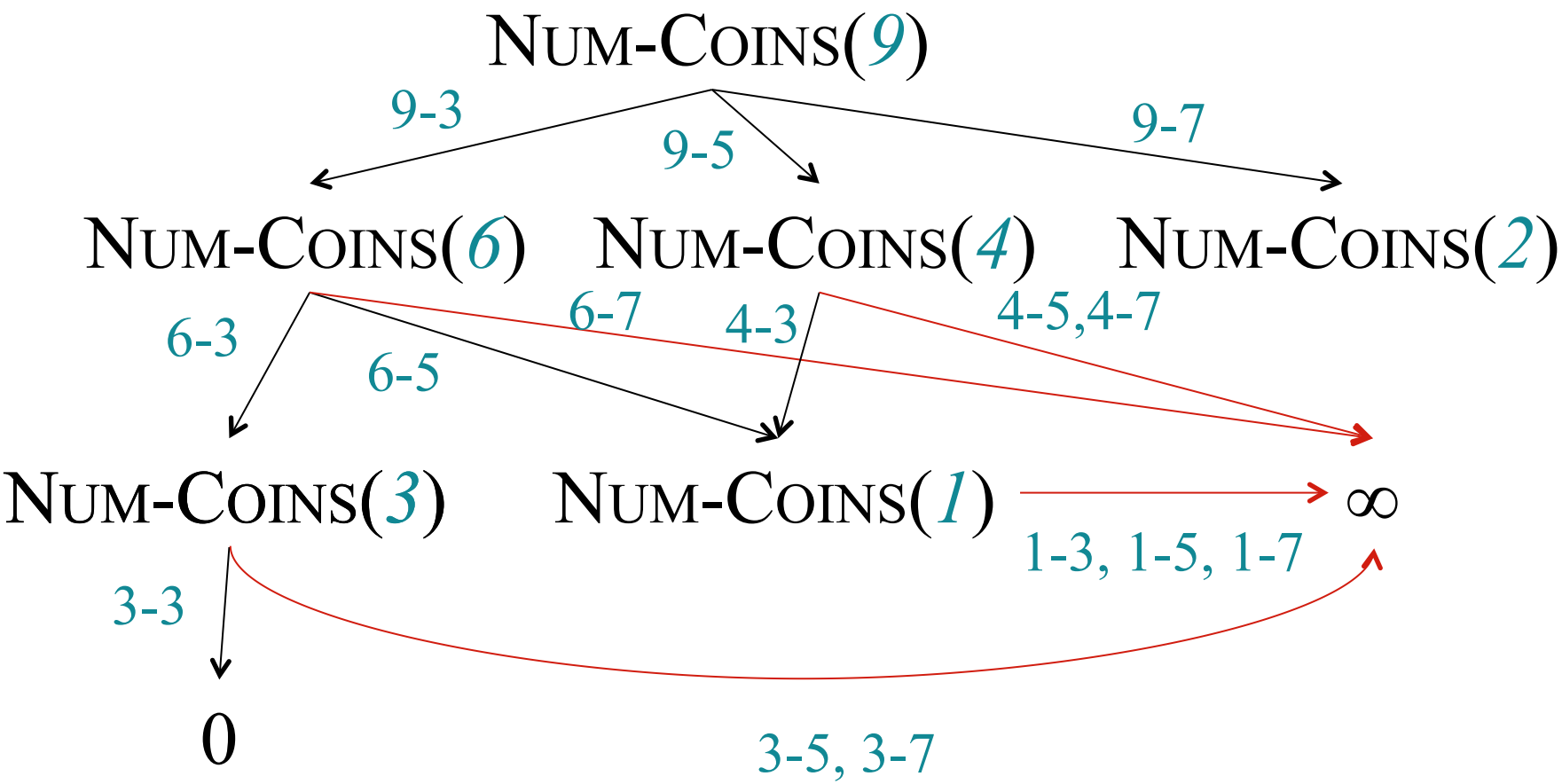
Example: $n = 9$,

denominations 3, 5, 7



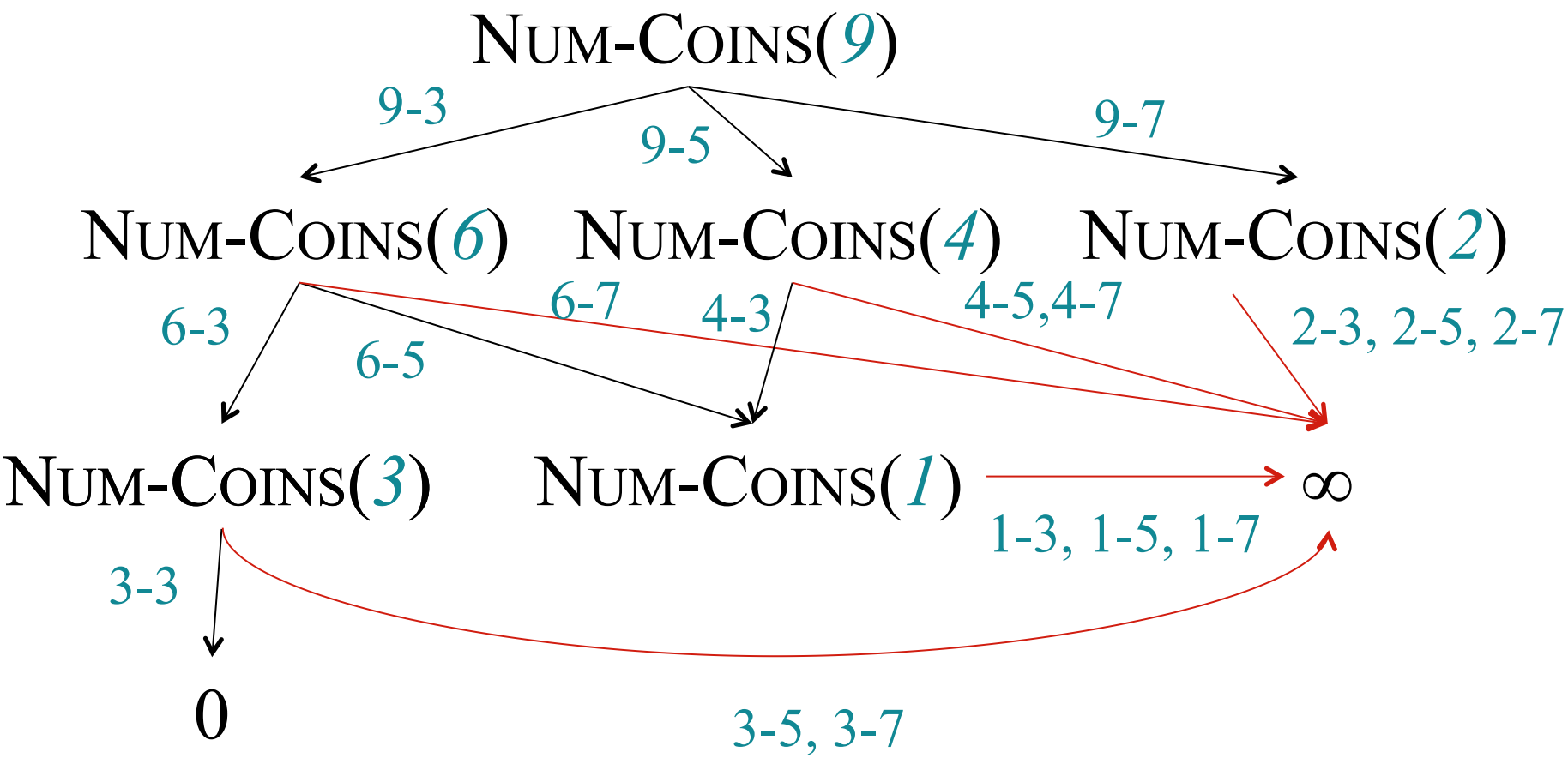
Example: $n = 9$,

denominations 3, 5, 7



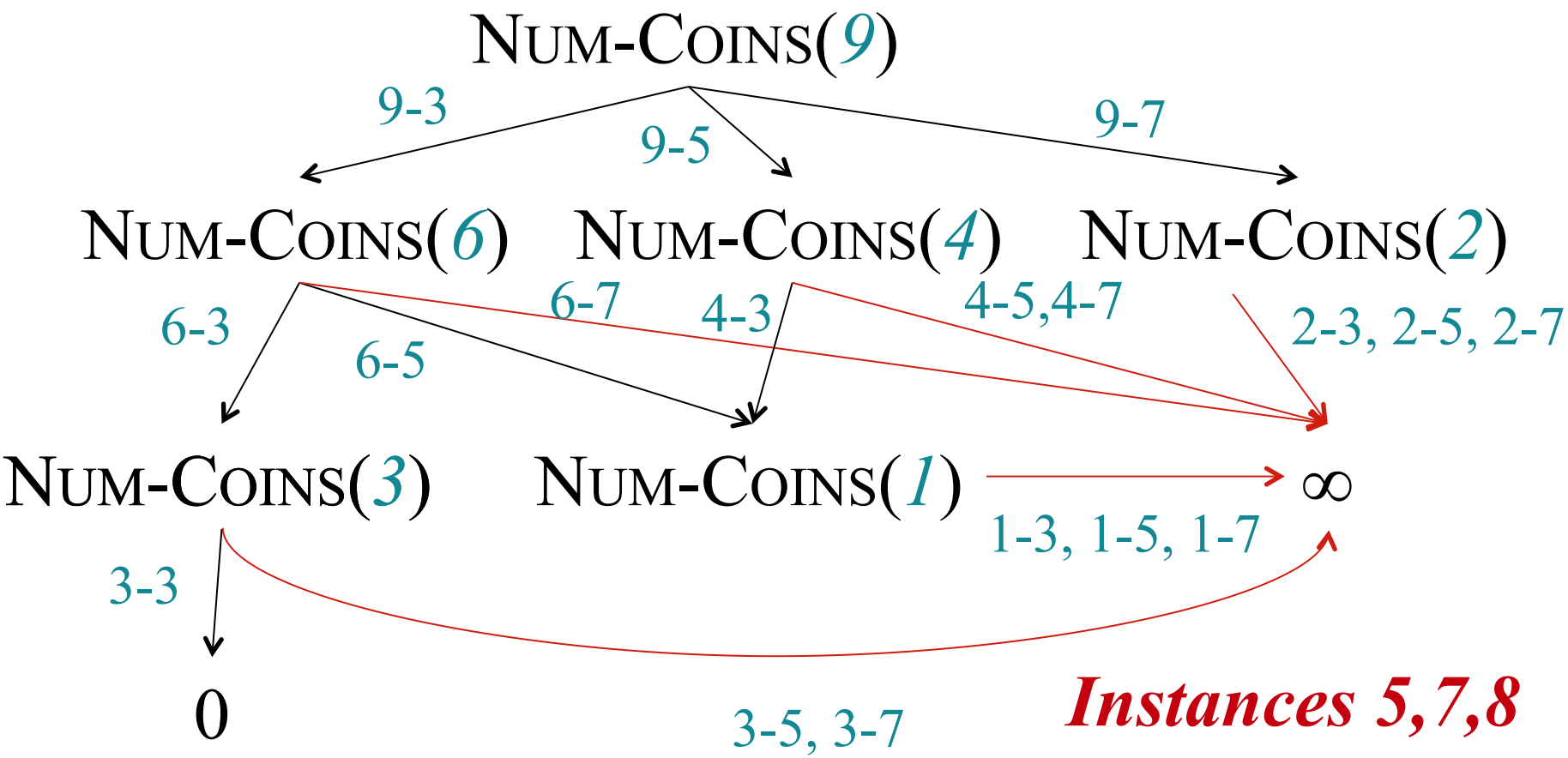
Example: $n = 9$,

denominations 3, 5, 7



Example: $n = 9$,

denominations 3, 5, 7



Instances 5,7,8 never solved!

Dynamic Programming

NUM-COINS-DP(n)

for $i \leftarrow 0$ **to** n

  $M[i] \leftarrow \infty$

$M[0] \leftarrow 0$

for $i \leftarrow 1$ **to** n

  for $j \leftarrow 1$ **to** k

    if $(i\text{-}d_j \geq 0)$ **and** $(M[i\text{-}d_j]+1 < M[i])$ **then**

      $M[i] \leftarrow M[i\text{-}d[j]] + 1$

Runtime $\Theta(nk)$

All subproblems solved.

Recap (Coin Changing)

- Greedy algorithm does not give optimal solution to coin changing
- Satisfies optimal substructure property – recursive solution
- Overlapping subproblems, $\Theta(n)$ problems
- Memoized solution – some subproblems do not need to be solved

Analysis and Design of Algorithms

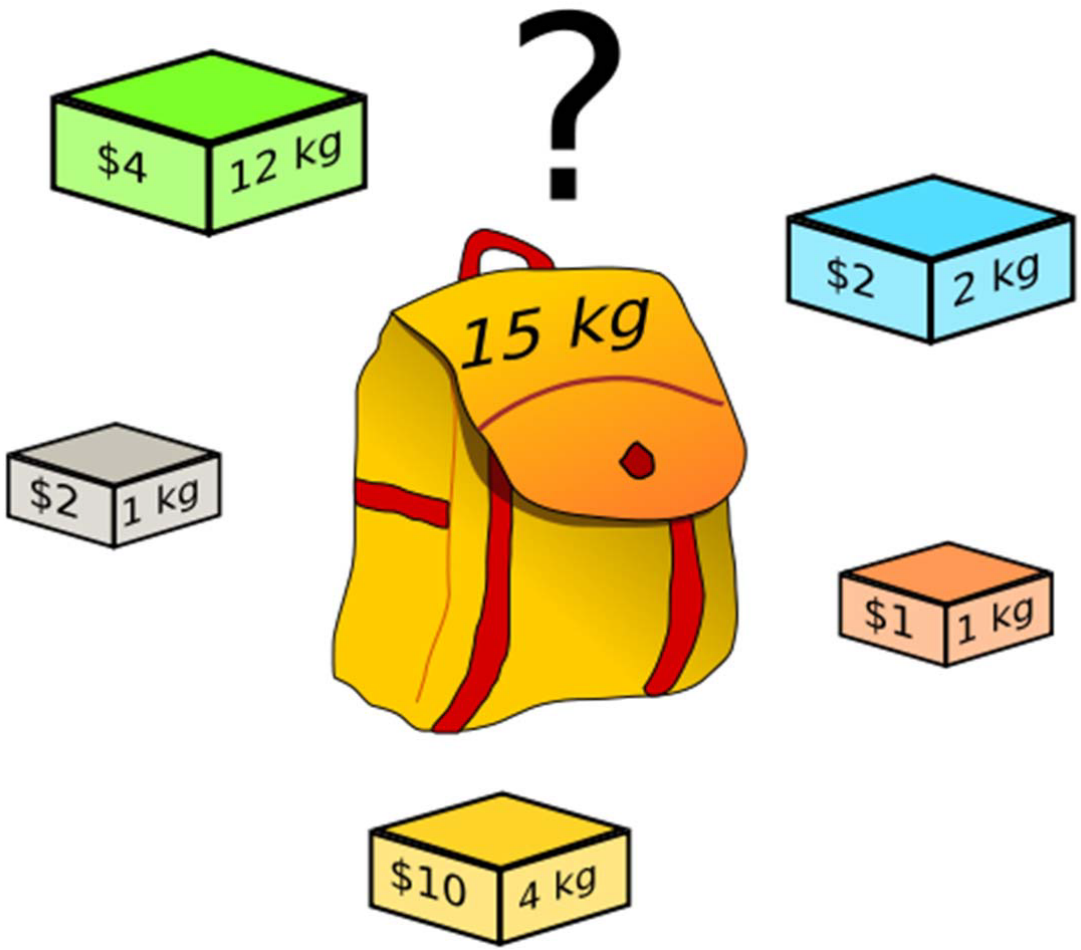
CS3230

Dynamic Programming Examples

- Coin changing
- **Knapsack problem**
- Matrix chain multiplication

Prof. Lee Wee Sun

Knapsack Problem



What is the maximum value you can get?

$15

KNAPSACK

Input:

$(w_1, v_1), (w_2, v_2), \ldots, (w_n, v_n)$, and W

Output: A subset $S \subseteq \{1, 2, \ldots, n\}$ that maximizes

$\Sigma_{i \in S} v_i$ such that $\Sigma_{i \in S} w_i \leq W$

Simple Slow Solution

KNAPSACK($\{v_1, v_2, \ldots, v_n\}$, $\{w_1, w_2, \ldots, w_n\}$, W)

$maxV = 0$

for every subset $S \subseteq \{1, 2, \ldots, n\}$

if $\Sigma_{i \in S} w_i \leq W$ and $\Sigma_{i \in S} v_i > maxV$ **then**

$maxV = \Sigma_{i \in S} v_i$

return $maxV$

Running time:

There are 2^n possible subsets S of $\{1, 2, \ldots, n\}$

It takes $O(n)$ time to compute $\Sigma_{i \in S} w_i$ and $\Sigma_{i \in S} v_i$

Hence, the running time is $O(n2^n)$

Decision version of **KNAPSACK** is NP-complete!
Any efficient algorithm?

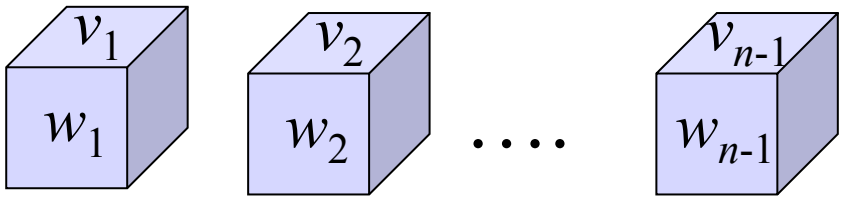
Dynamic Programming

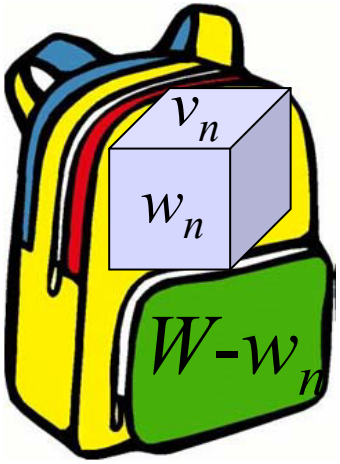
Problem: $(w_1, v_1), \ldots, (w_n, v_n), W$

Is there optimal substructure?

Case 1: Item n (the last one) is taken

Have optimal solution to subproblem defined by $(w_1, v_1), \ldots, (w_{n-1}, v_{n-1}), W\text{-}w_n$

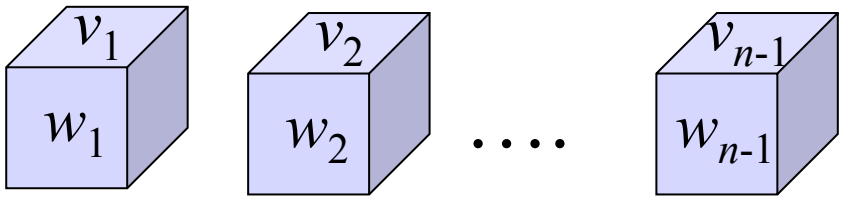




Otherwise, by *cut and paste* argument, we can get a better solution

Case 2: Item n (the last one) is **not** taken

Have optimal solution to subproblem defined by $(w_1, v_1), \ldots, (w_{n-1}, v_{n-1}), W$



Otherwise, by *cut and paste* argument, we can get a better solution

Recursive solution:

Let $m[i,w]$ be the maximum value that can be obtained using a subset of items in $\{1,2,..,i\}$ with total weight no more than w. Then

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

KNAPSACK(v, w, W)

for $j \leftarrow 0$ **to** W ▷ Initialization

  $m[0, j] \leftarrow 0$

for $i \leftarrow 1$ **to** n ▷ Initialization

  $m[i, 0] \leftarrow 0$

▷ Recursive cases

…

return $m[n, W]$

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

▹ Recursive cases

Runtime: $\Theta(nW)$

for $i \leftarrow 1$ **to** n

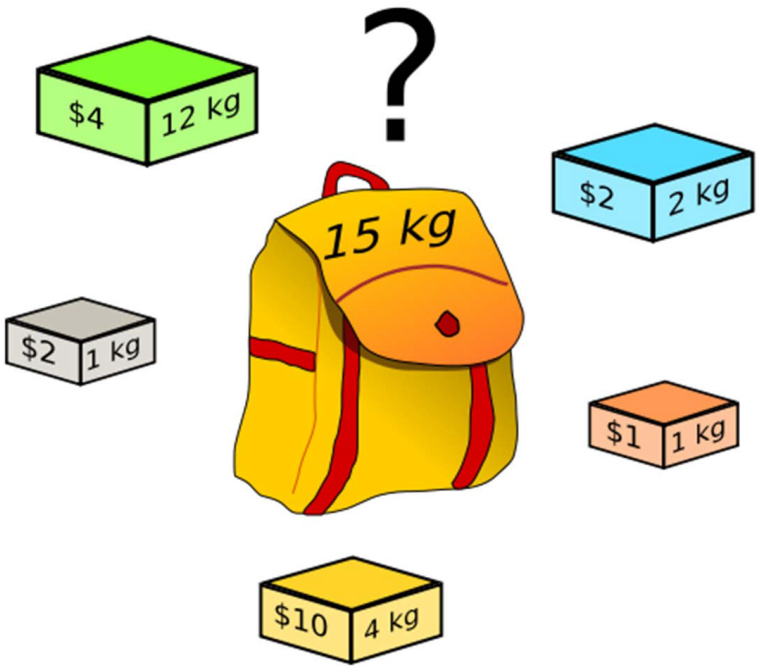
  for $j \leftarrow 0$ **to** W

    if $j \geq w[i]$ **then**

      $m[i, j] \leftarrow \max\{m[i\text{-}1, j\text{-}w[i]] + v[i], m[i\text{-}1, j]\}$

    else

      $m[i, j] \leftarrow m[i\text{-}1, j]$



$v[1..5] = \{4, 2, 10, 1, 2\}$

$w[1..5] = \{12, 1, 4, 1, 2\}$

$W = 15$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0															
2	0															
3	0															
4	0															
5	0															

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

$$v[1..5] = \{4, 2, 10, 1, 2\}$$

$$w[1..5] = \{12, 1, 4, 1, 2\}$$

$$W = 15$$

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	**4**			
2	0															
3	0															
4	0															
5	0															

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

$$v[1..5] = \{4, 2, 10, 1, 2\}$$

$$w[1..5] = \{12, 1, 4, 1, 2\}$$

$$W = 15$$

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	**12**										
4	0															
5	0															

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

$$v[1..5] = \{4, 2, 10, 1, 2\}$$

$$w[1..5] = \{12, 1, 4, 1, 2\}$$

$$W = 15$$

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
4	0	2	3	3	10	**12**										
5	0															

$$m[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ \max\{m[i-1, w-w_i] + v_i, m[i-1,w]\} & \text{if } w_i \le w \\ m[i-1,w] & \text{otherwise} \end{cases}$$

$$v[1..5] = \{4, 2, 10, 1, 2\}$$

$$w[1..5] = \{12, 1, 4, 1, 2\}$$

$$W = 15$$

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
3	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
4	0	2	3	3	10	12	13	13	13	13	13	13	13	13	13	13
5	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	**15**

PRINT-SET(n,ω)

if $\omega \geq w[i]$ **and**

$m[i,\omega] = m(i-1, \omega-w[i])+v[i]$ **then**

return PRINT-SET($i-1, \omega - w[i]$) $\cup \{i\}$

else

return PRINT-SET($i-1, \omega$)

$v[1..5] = \{4, 2, 10, 1, 2\}$

$w[1..5] = \{12, 1, 4, 1, 2\}$

$W = 15$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
✔ **2**	0	2	2	2	2	2	2	2	2	2	2	2	4	6	6	6
✔ **3**	0	2	2	2	10	12	12	12	12	12	12	12	12	12	12	12
✔ **4**	0	2	3	3	10	12	13	13	13	13	13	13	13	13	13	13
✔ **5**	0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	**15**

Recap (Knapsack)

- Optimal substructure and recursive solution in terms of set of items from 1 to i
- $\Theta(nW)$ subproblems, dynamic programming with runtime $\Theta(nW)$

Analysis and Design of Algorithms

CS3230

Dynamic Programming Examples

- Coin changing
- Knapsack problem
- **Matrix chain multiplication**

Prof. Lee Wee Sun

Matrix Chain Multiplication

Different ***parenthesization*** of a chain of matrix for multiplication can have dramatic impact on the amount of computation required.

Example: $((A_1A_2)A_3)$ and $(A_1(A_2A_3))$ are two different parenthesization of the same matrix chain multiplication $A_1A_2A_3$.

Consider A_1, A_2, A_3 of sizes 10 x 100, 100 x 5 and 5 x 50 respectively:

- $((A_1A_2)A_3)$ cost 10*100*5=5000 scalar mult followed by 10*5*50=2500 scalar mult for total of **7500** mult.
- $(A_1(A_2A_3))$ cost 100*5*50=25000 followed by another 10*100*50=50000 for a total of **75000** scalar multiplications.

Matrix-chain multiplication problem:

Given a chain $<A_1, A_2, \ldots, A_n>$ of n matrices, where for $i = 1, 2, \ldots, n$, the matrix A_i has dimension $p_{i-1} \times p_i$, parenthesize the product $A_1 A_2 \ldots A_n$ in a way that minimizes the number of scalar multiplications.

Optimal substructure:

Assume optimal *parenthesization* splits product $A_i A_{i+1} \ldots A_j$ between A_k and A_{k+1}, where $i \leq k < j$.

Then the parenthesizations of $A_i A_{i+1} \ldots A_k$ and $A_{k+1} \ldots A_j$ within the optimal parenthesization must also be optimal.

Proof:

Otherwise, by *cut and paste*, we can replace the parenthesization of $A_i A_{i+1} \ldots A_k$ or $A_{k+1} \ldots A_j$ by the optimal parenthesization and improve the optimal parenthesization of $A_i A_{i+1} \ldots A_j$: a contradiction.

Recursive solution:

Let $m[i,j]$ be the minimum cost for computing the matrix $A_{i..j}$. Then for computing $A_{i..k}A_{k+1..j}$:

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\} & \text{if } i < j. \end{cases}$$

Proof:

If $i = j$, cost is 0 as no multiplication required. Otherwise, the optimal split can occur at positions $i \leq k < j$. By the *optimal substructure property*, we need the optimal parenthesization for $A_i A_{i+1} \ldots A_k$ and $A_{k+1} \ldots A_j$.

Furthermore, $A_i A_{i+1} \ldots A_k$ has size $p_{i-1} \times p_k$ while $A_{k+1} \ldots A_j$ has size $p_k \times p_j$, so the optimal cost in this case is

$$\min_{i \leq k < j} \{m[i,k] + m[k+1, j] + p_{i-1} p_k p_j\}$$

MATRIX-CHAIN-ORDER(p)

$n \leftarrow \text{length}[p] - 1$

for $i \leftarrow 1$ **to** n ▷ Initialization

$m[i,i] \leftarrow 0$

▷ Main loop

….

▷ Return cost and reconstruction tables

return m and s

▹ Main loop, l is chain length

for $l \leftarrow 2$ **to** n

  for $i \leftarrow 1$ **to** $n - l + 1$

    $j \leftarrow i + l - 1$

    $m[i,j] \leftarrow \infty$

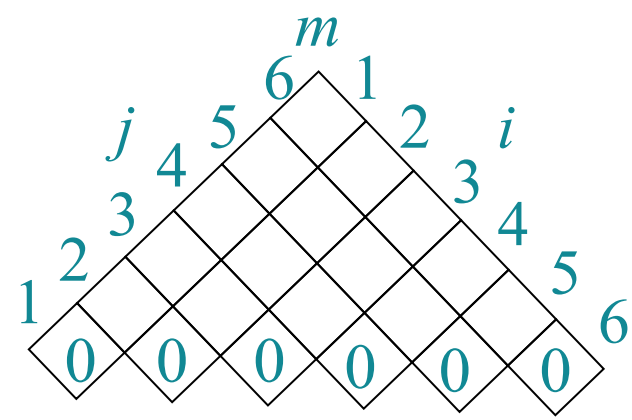
    for $k \leftarrow i$ **to** $j - 1$

      $q \leftarrow m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$

        if $q < m[i,j]$ **then**

          $m[i,j] \leftarrow q$

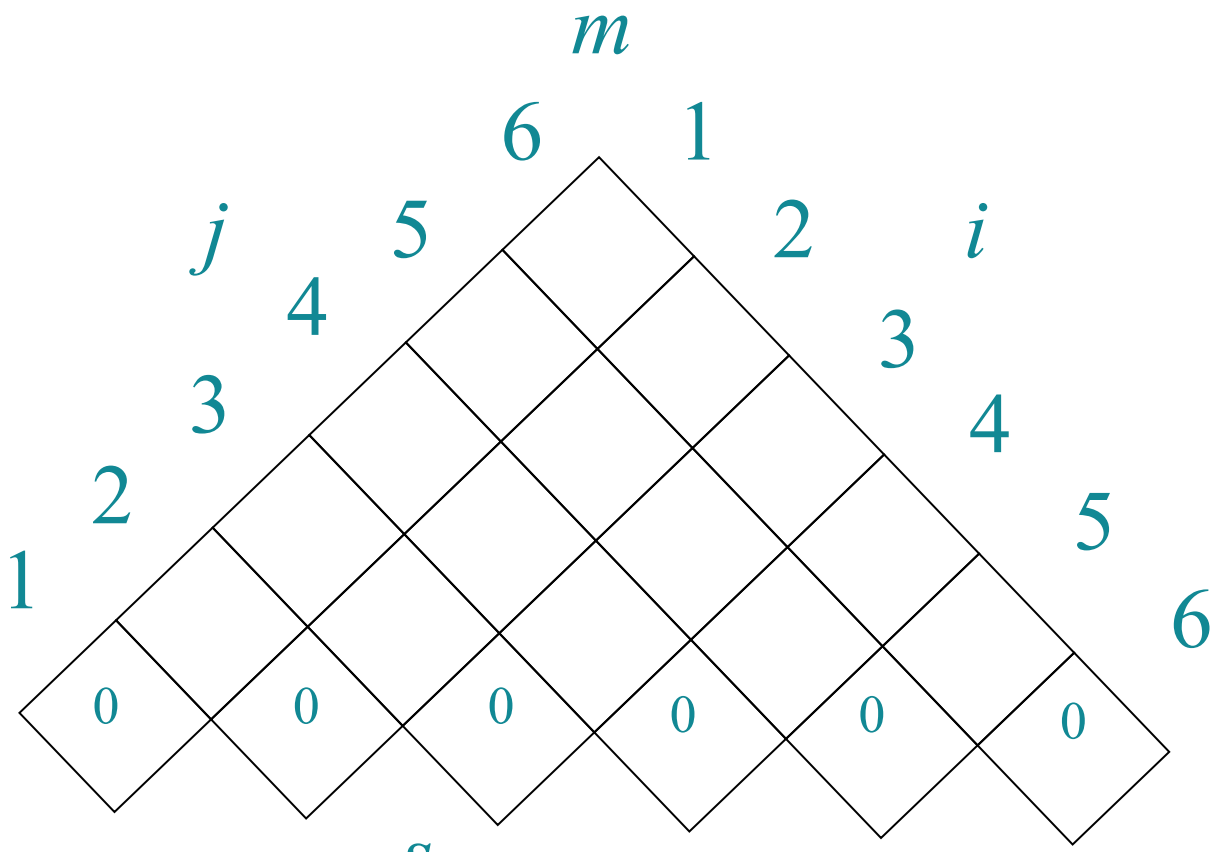
            $s[i,j] \leftarrow k$

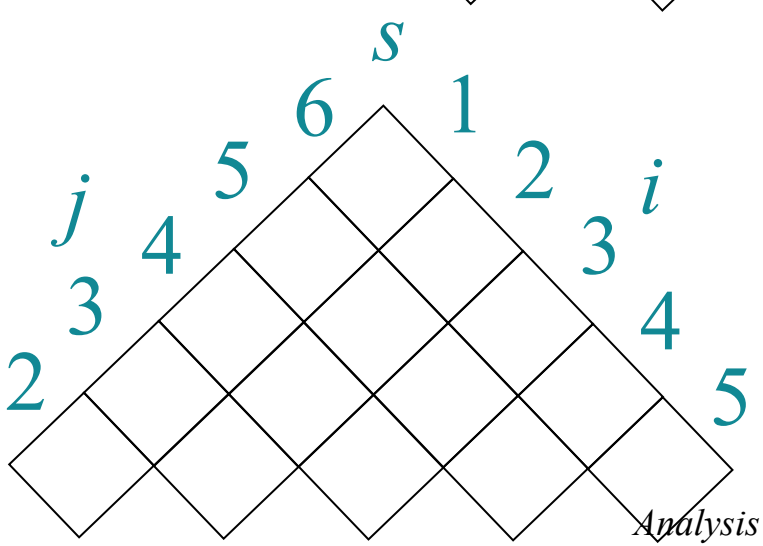


Overlapping subproblems:

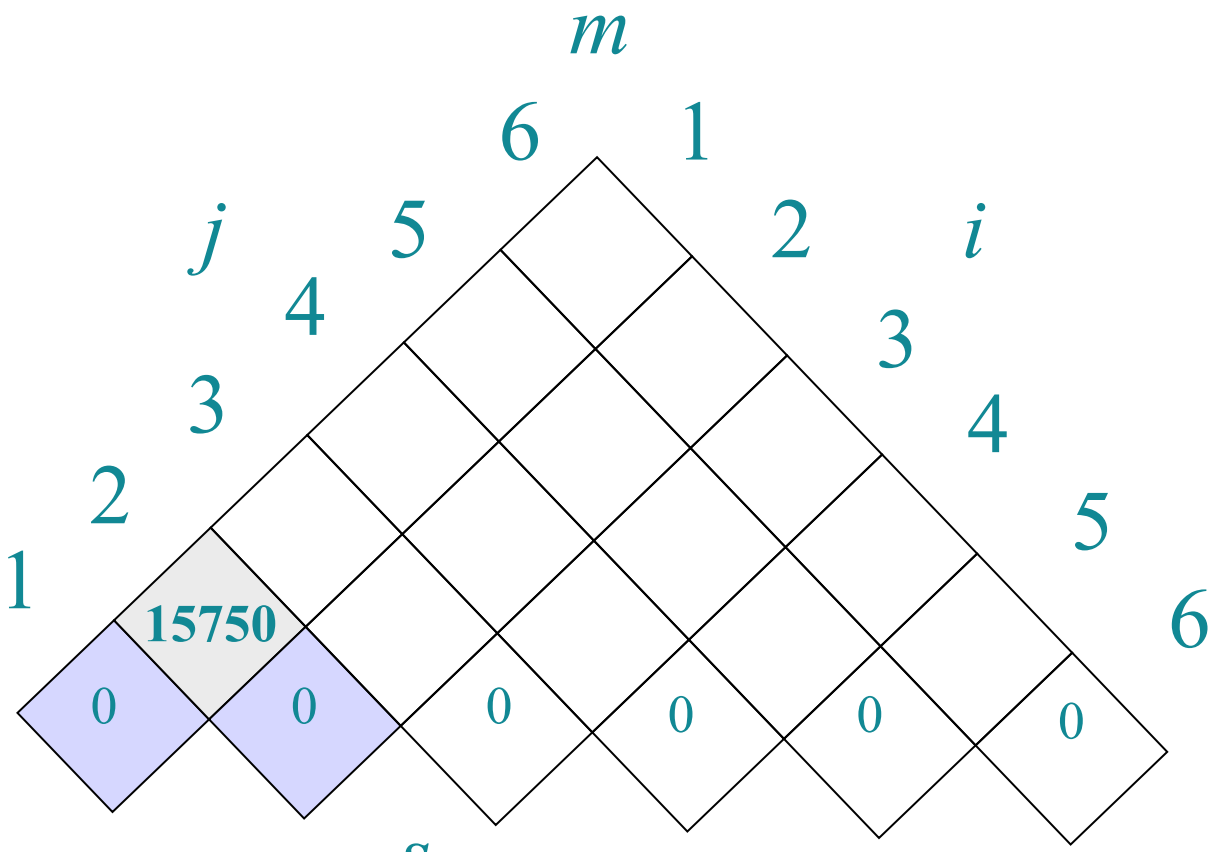
Only $\Theta(n^2)$ entries for $m[i,j]$.

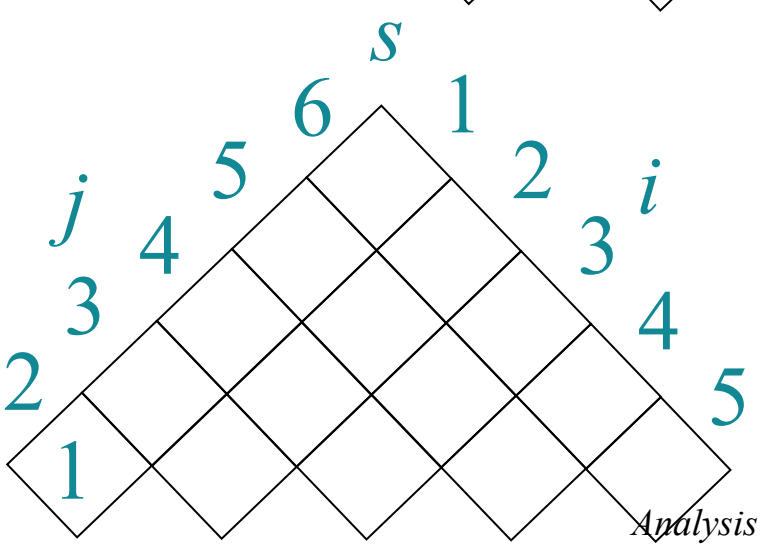
For each entry, $\Theta(n)$ computations,
so total runtime $\Theta(n^3)$



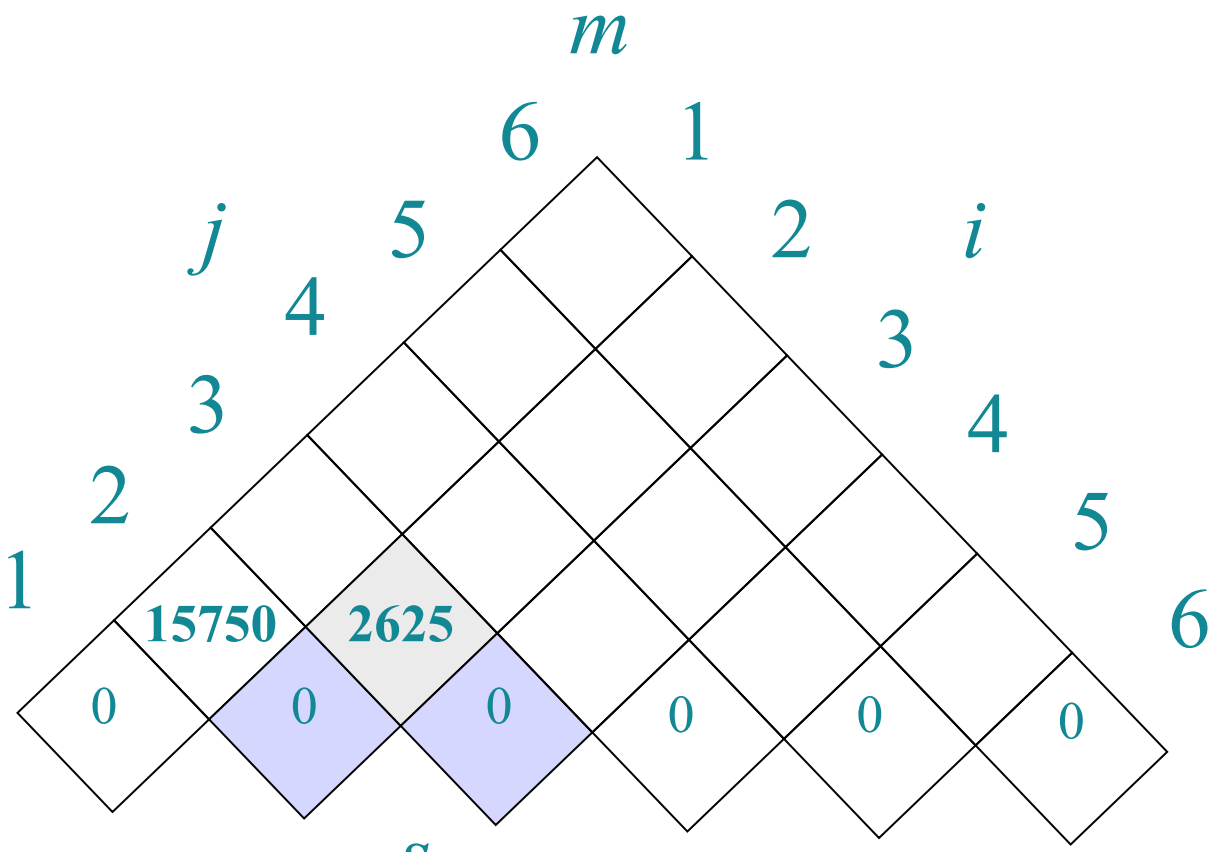


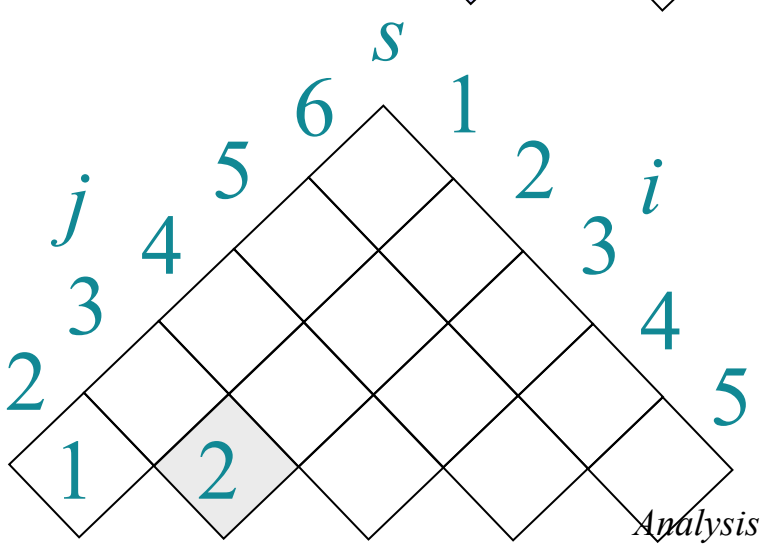
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



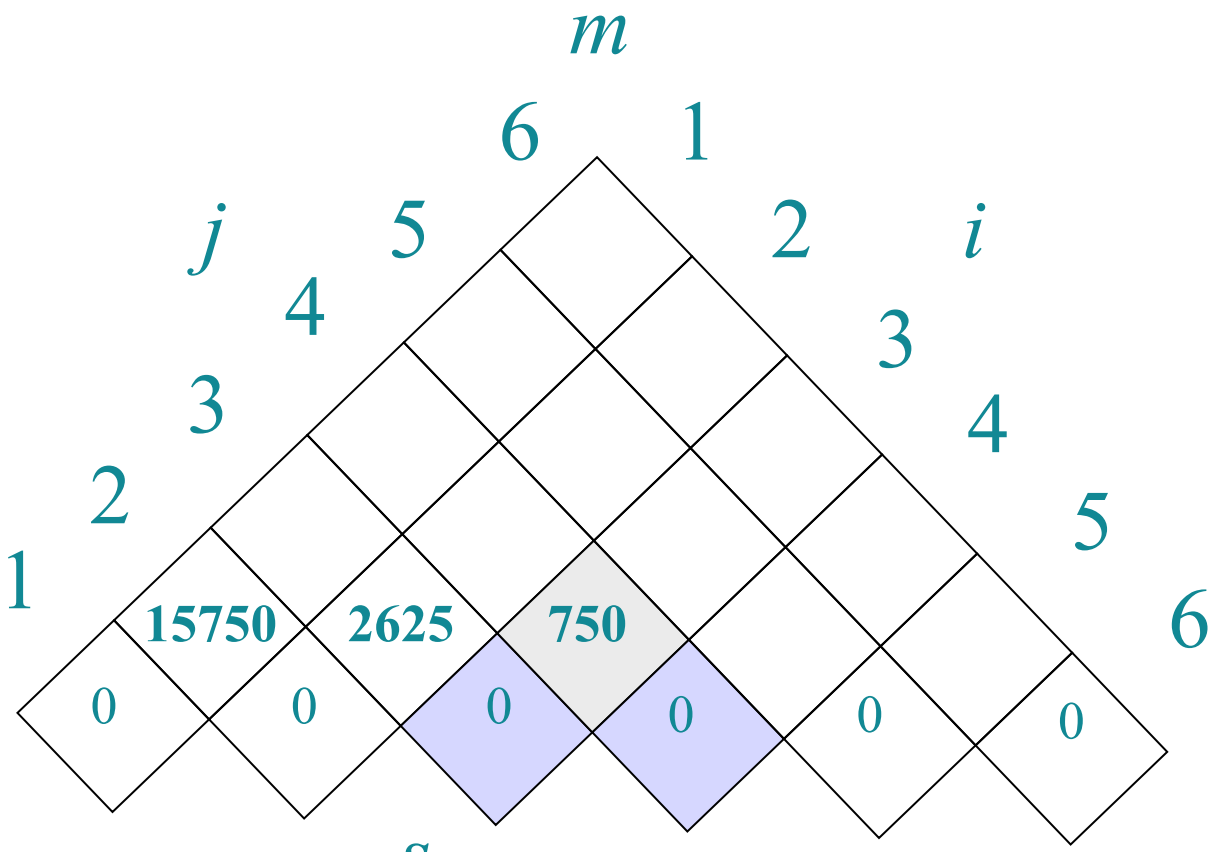


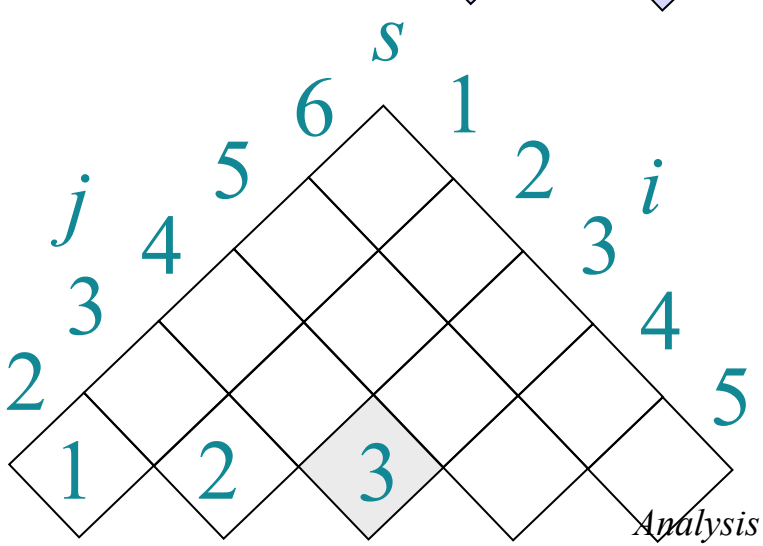
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



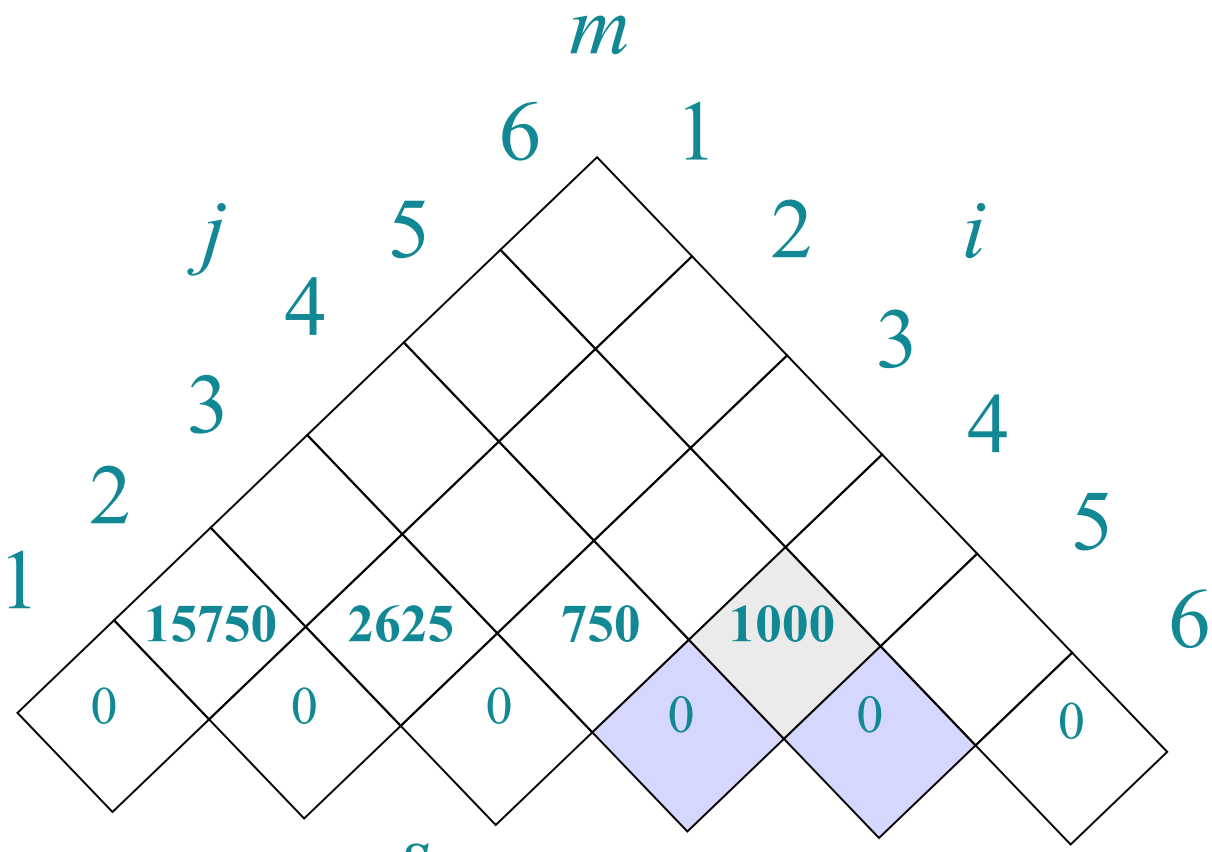


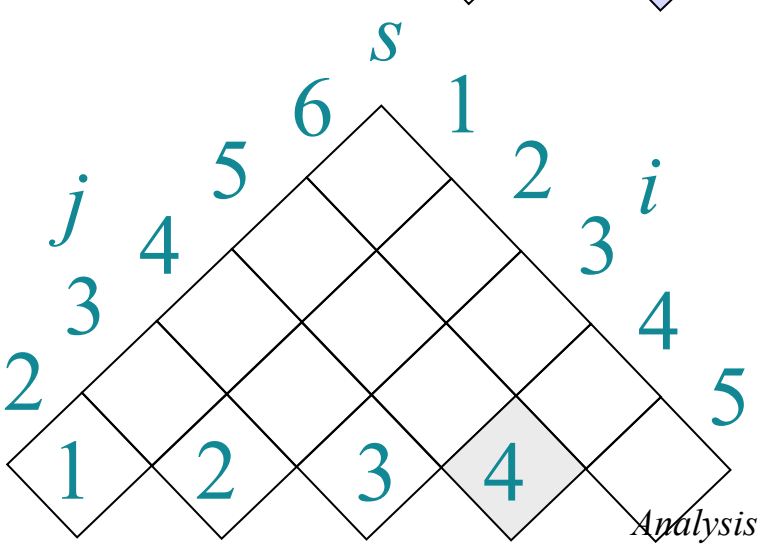
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



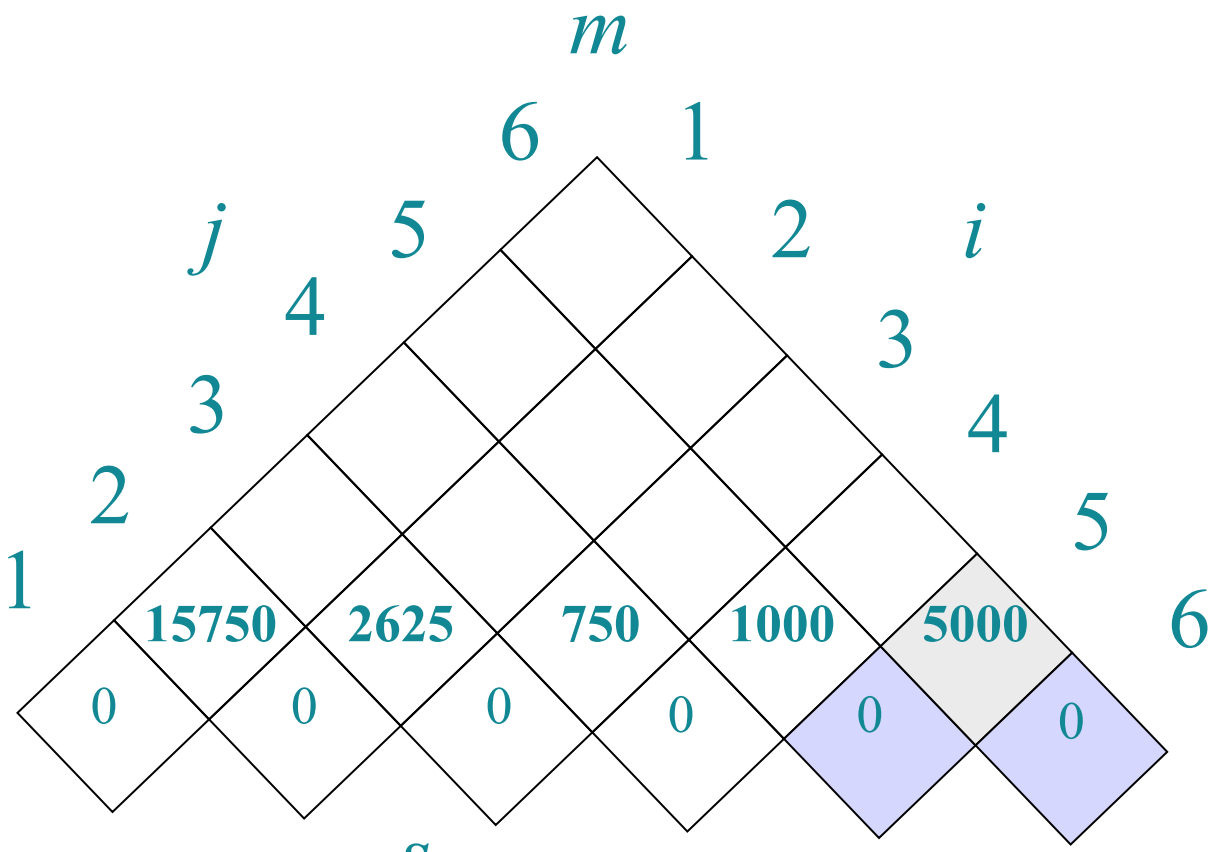


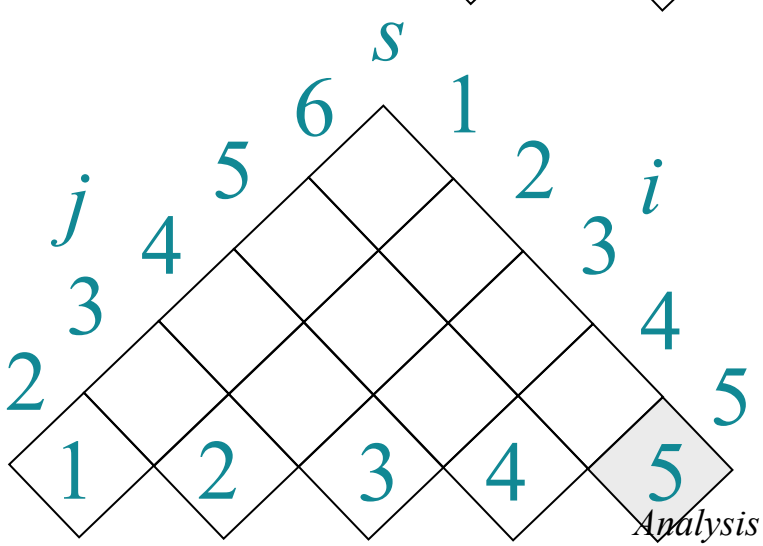
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



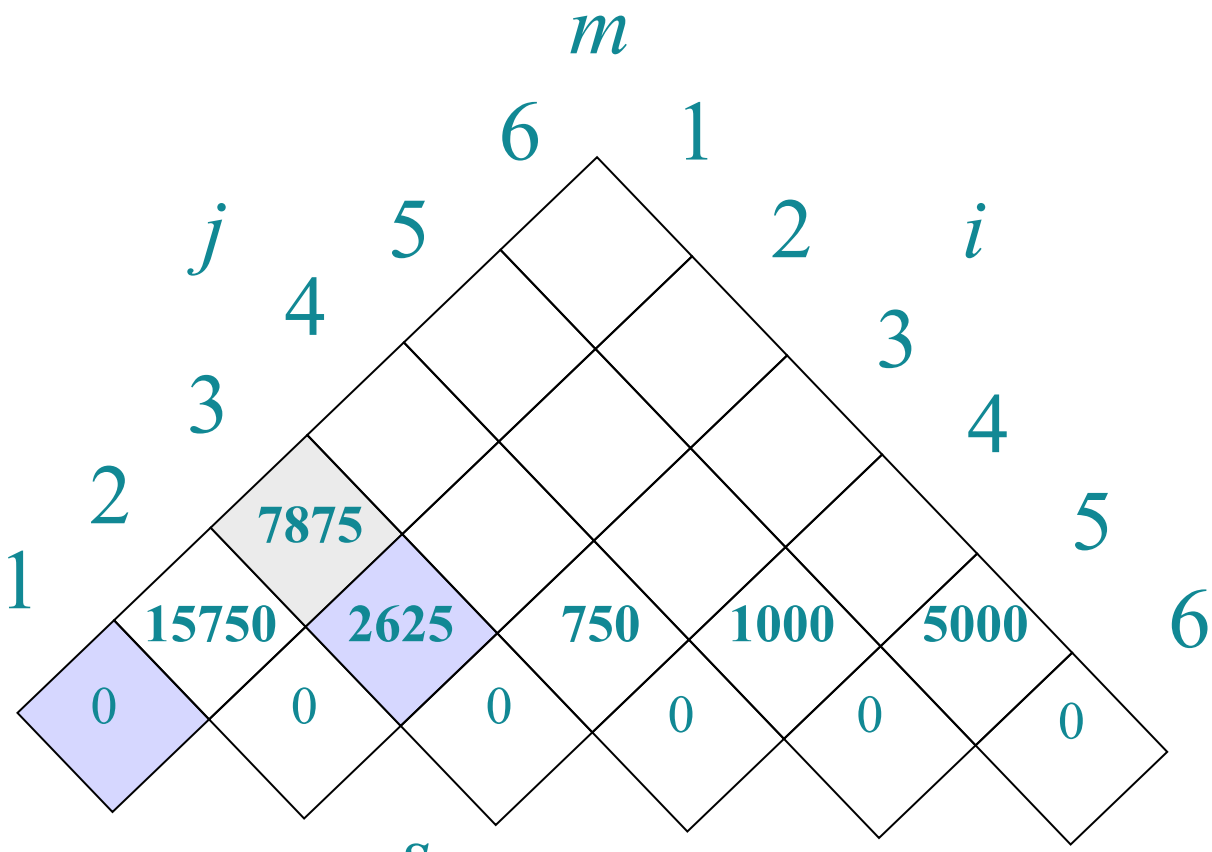


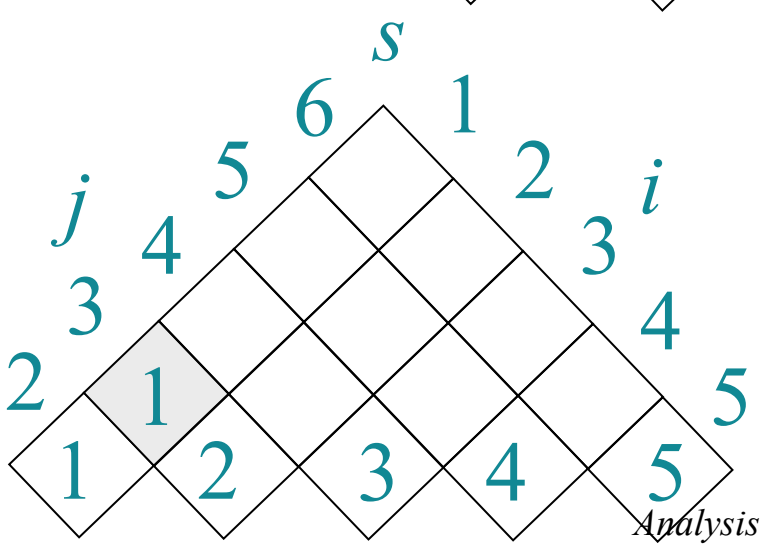
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



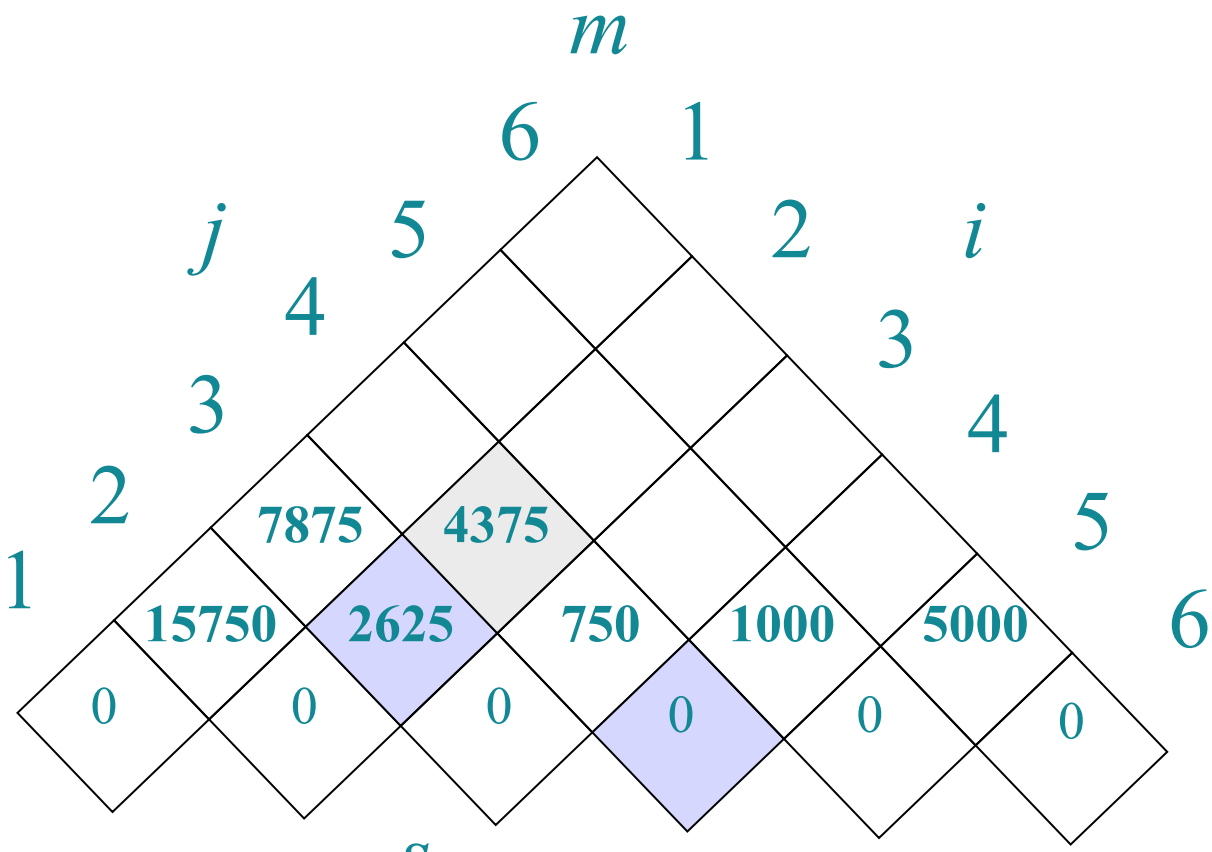


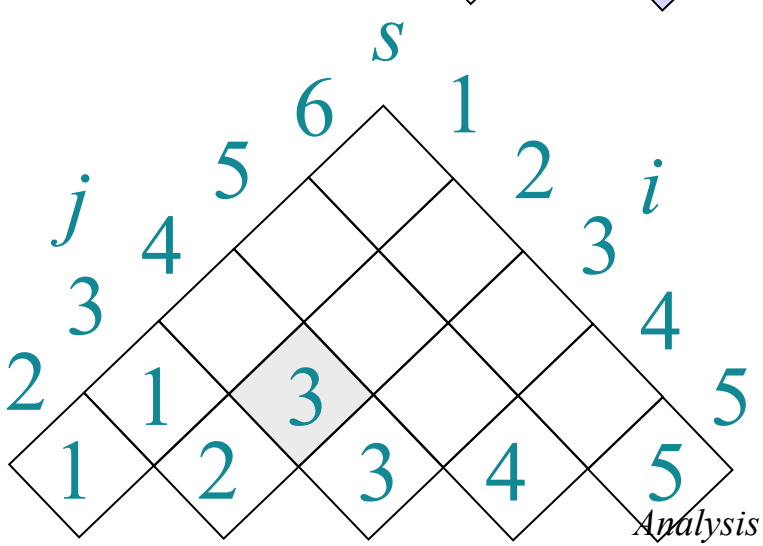
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



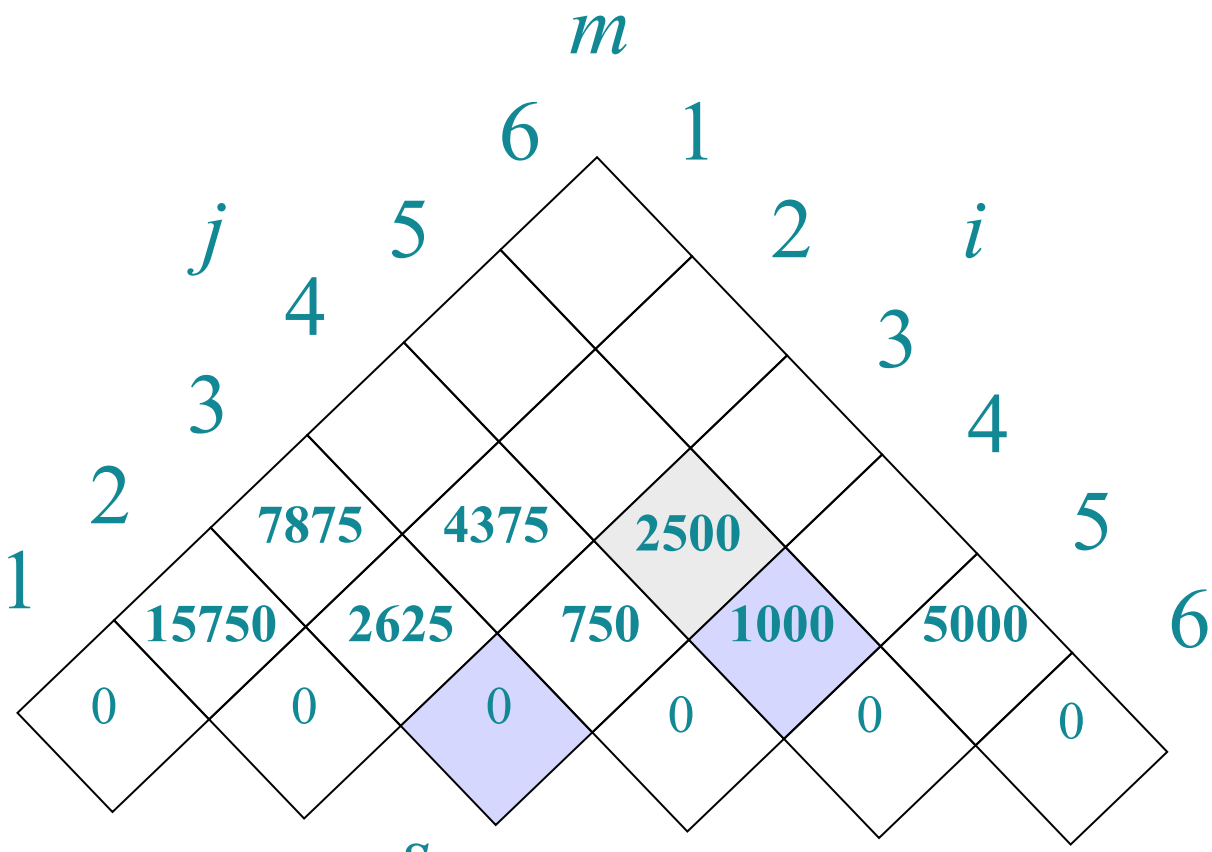


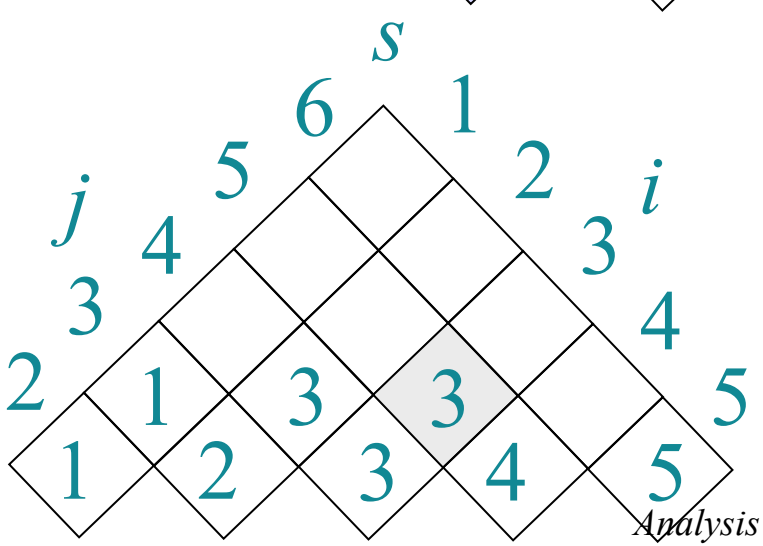
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



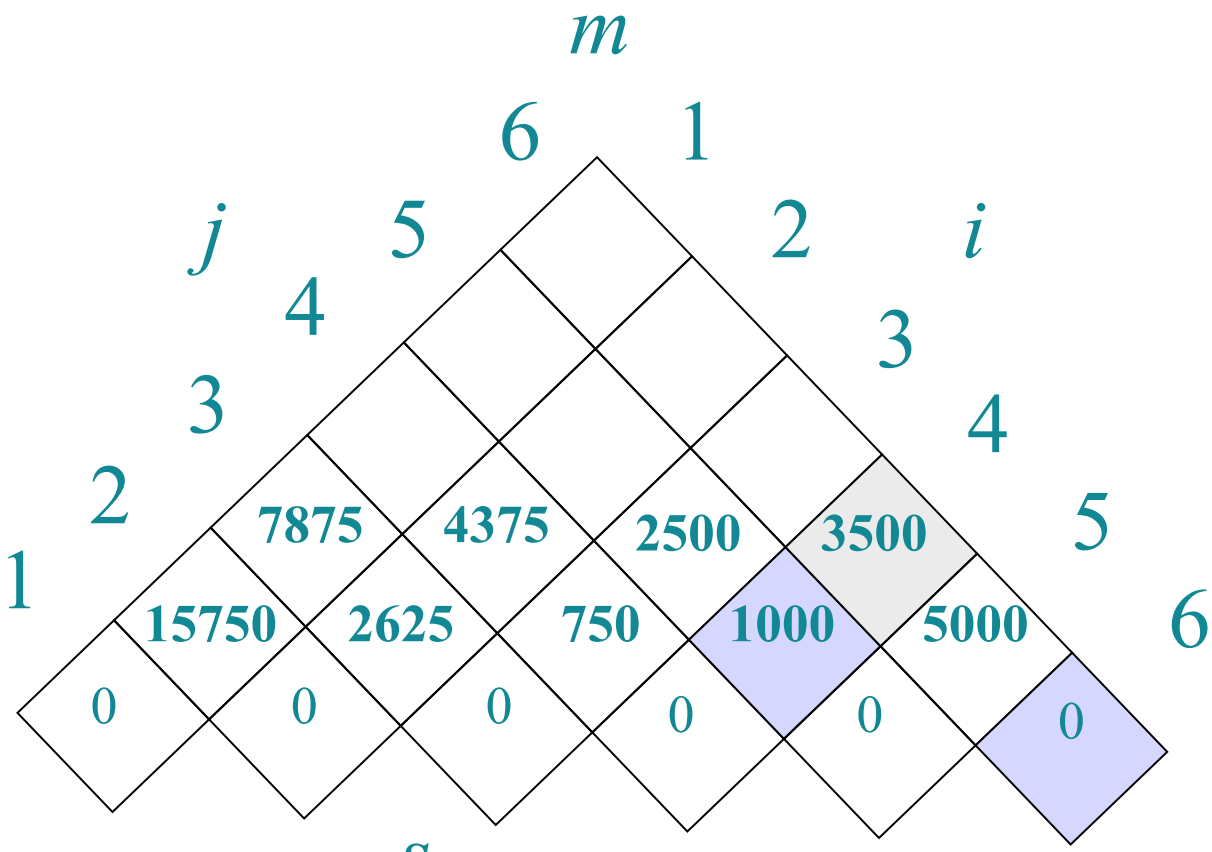


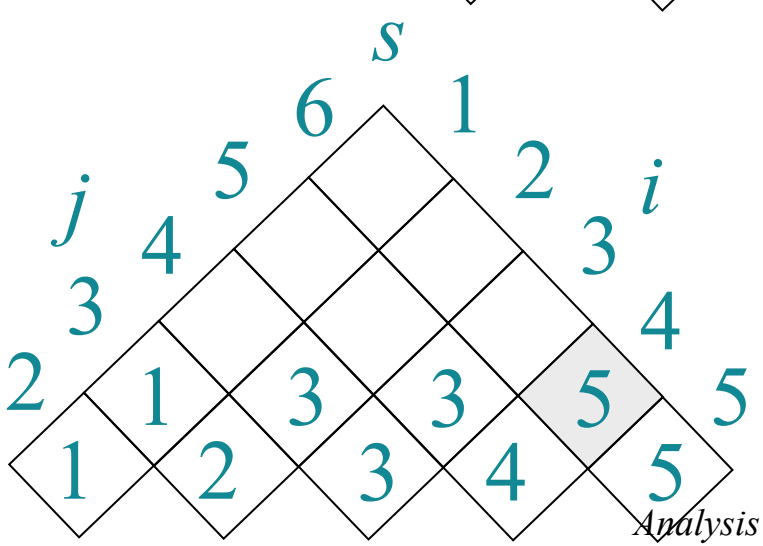
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



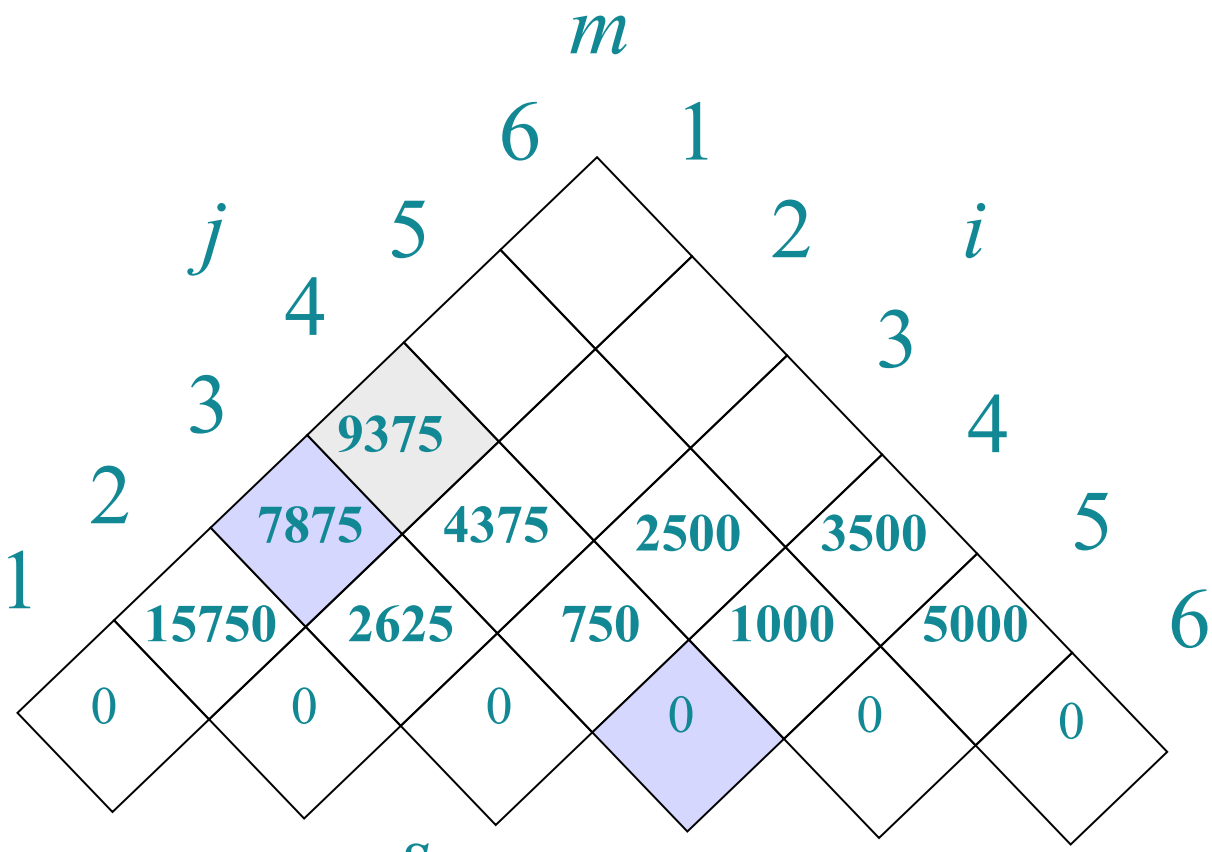


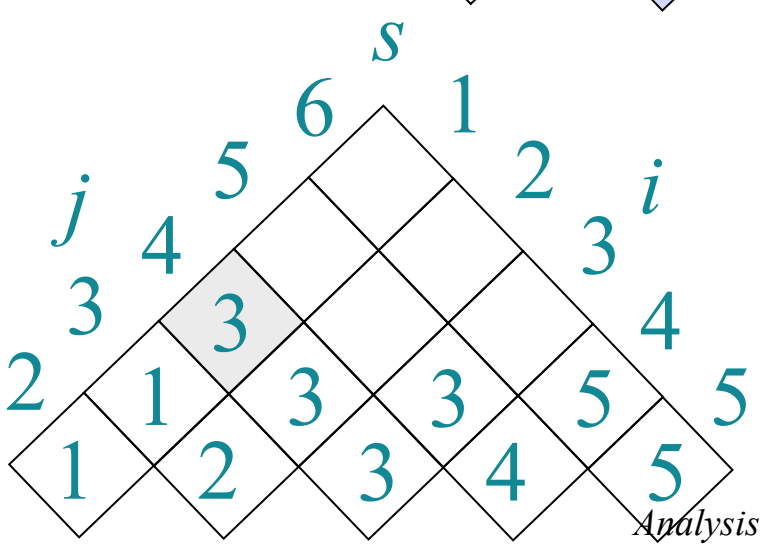
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



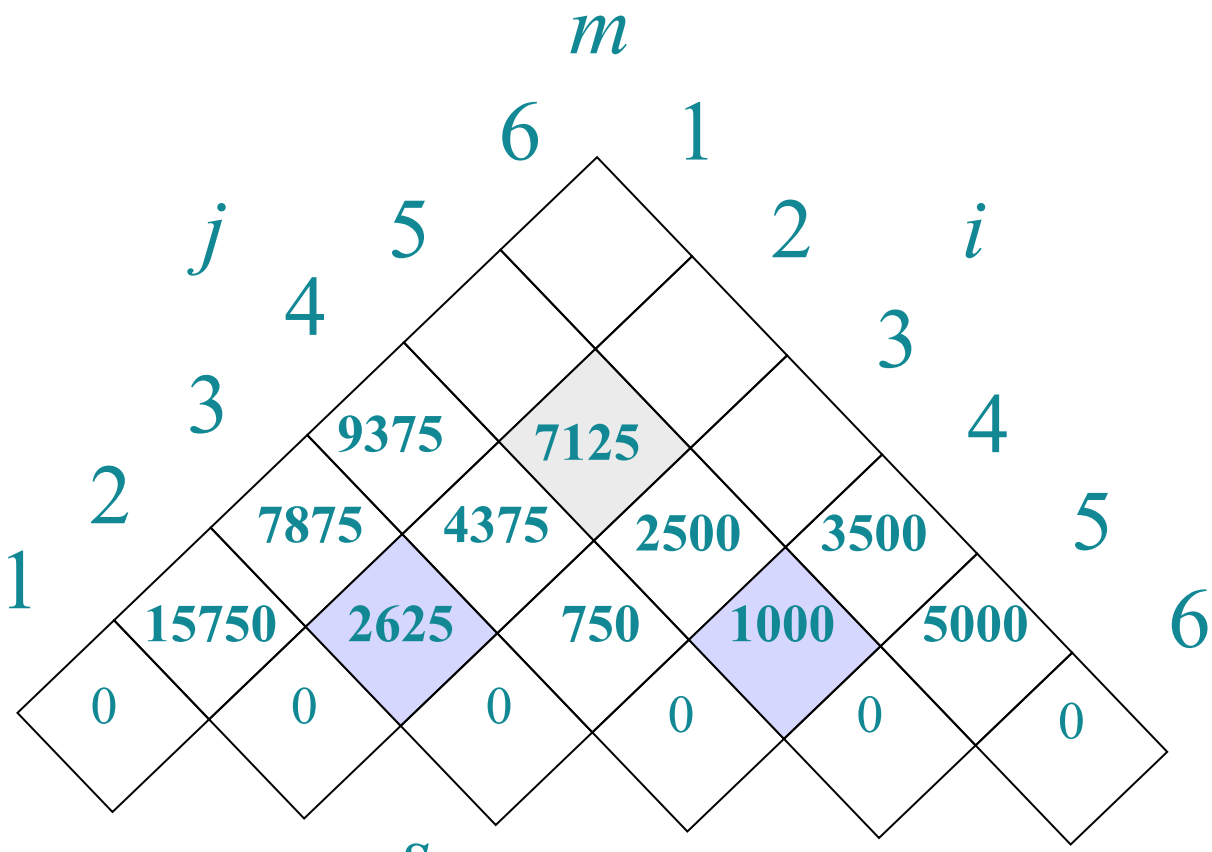


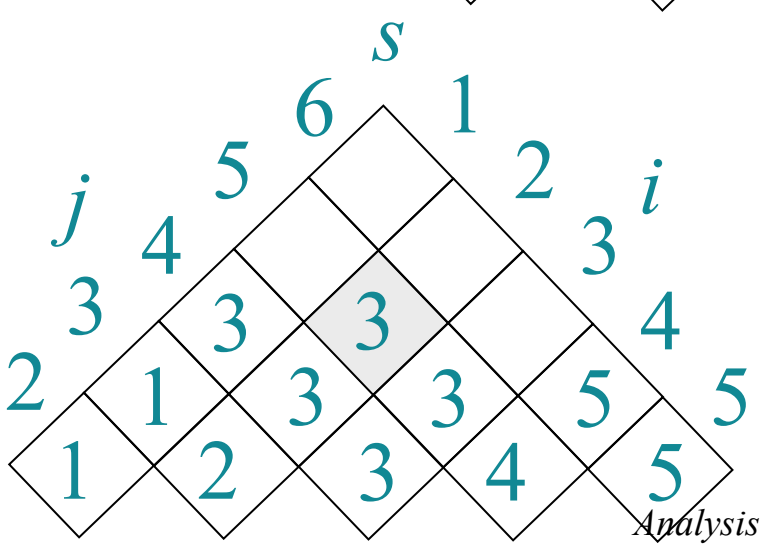
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



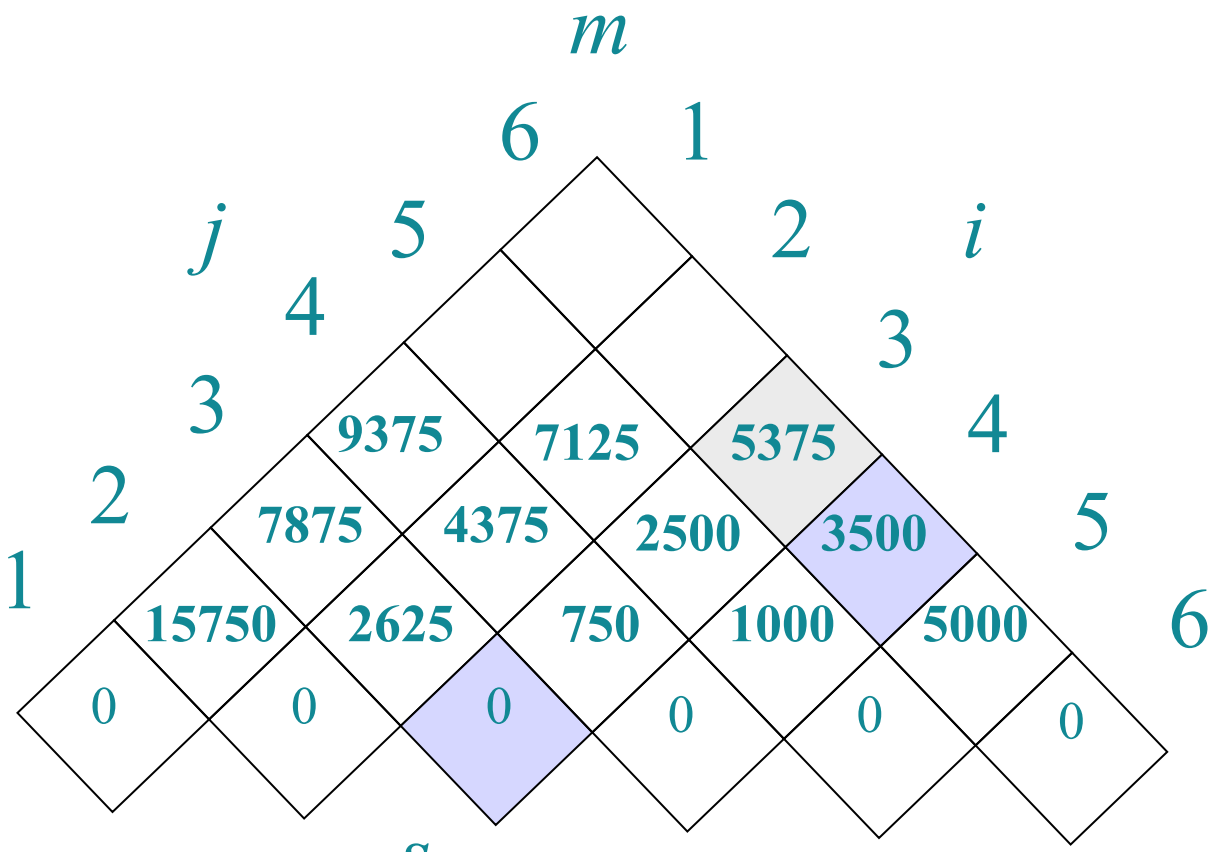


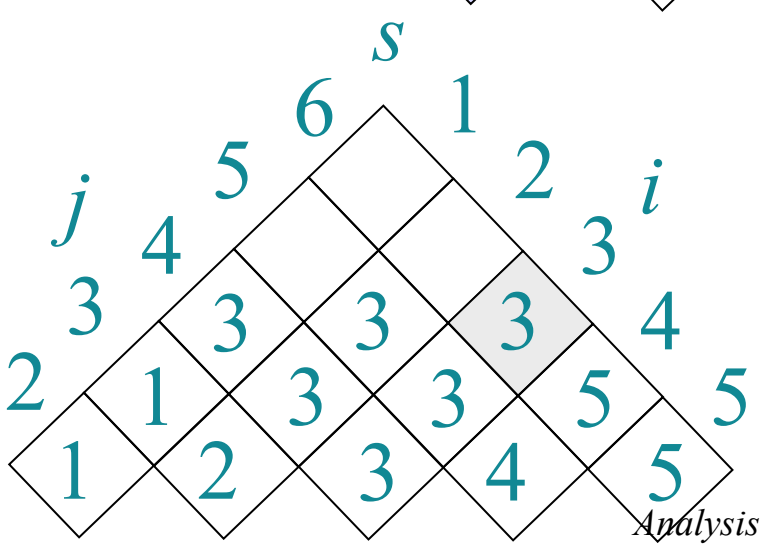
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



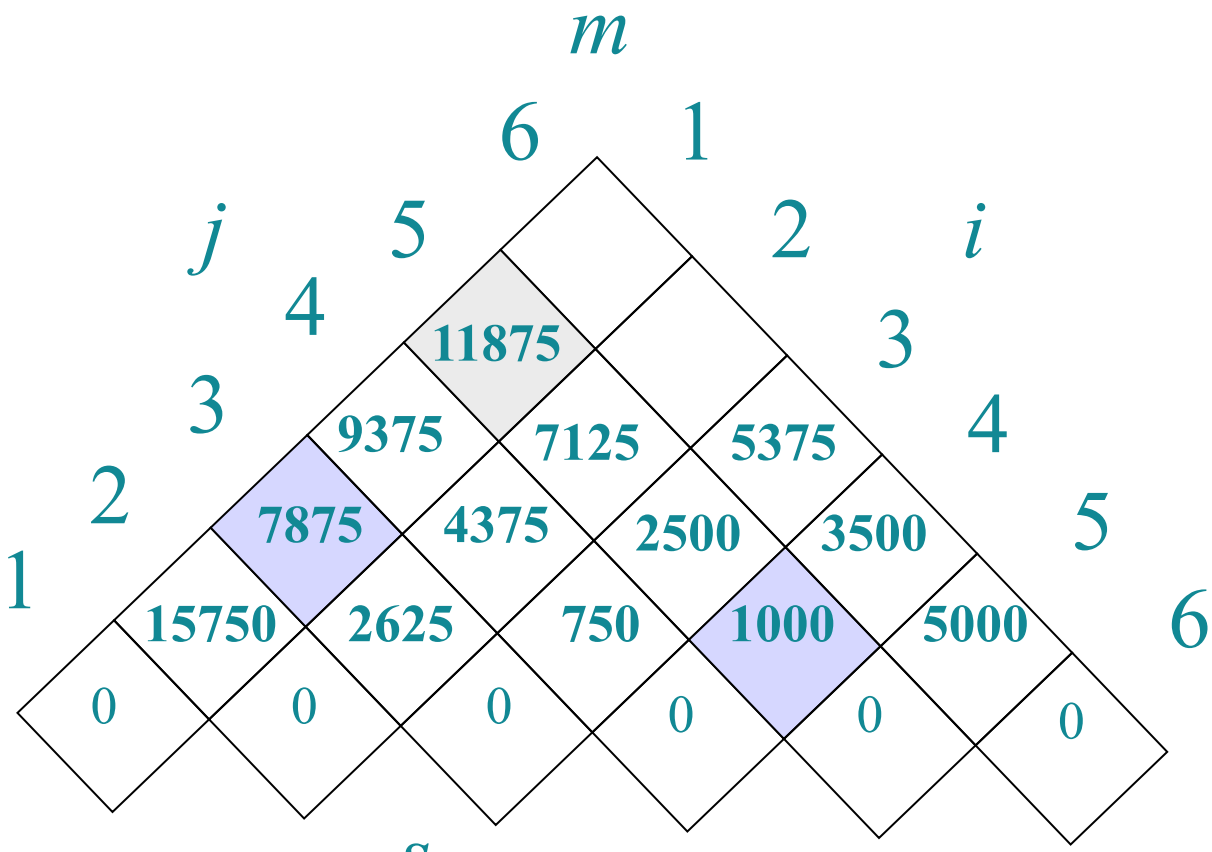


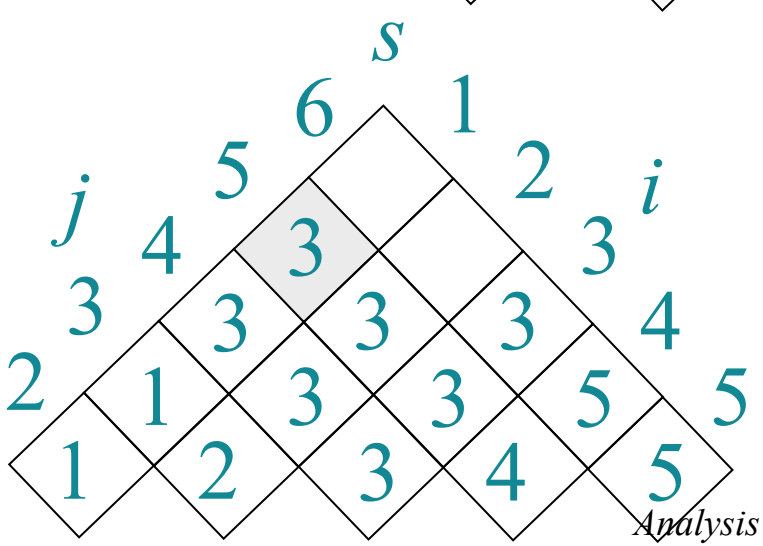
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



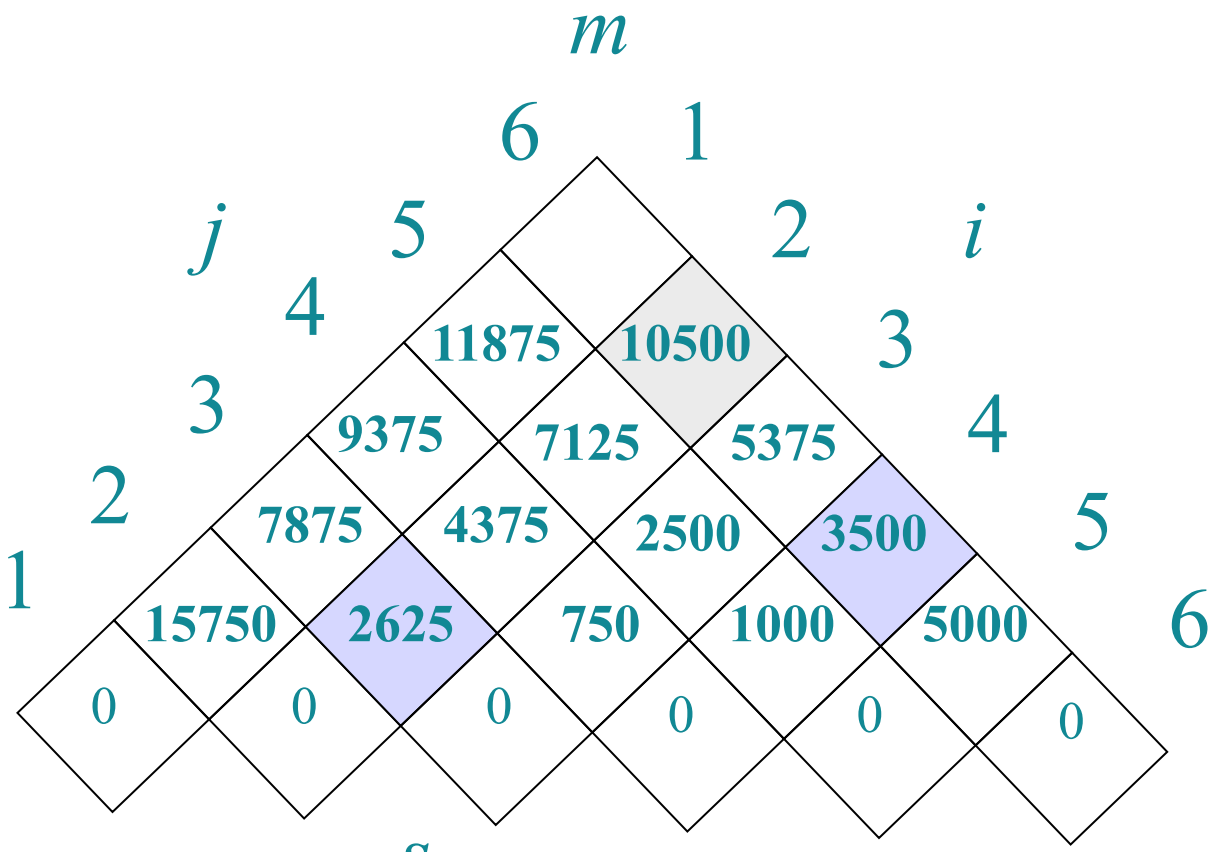


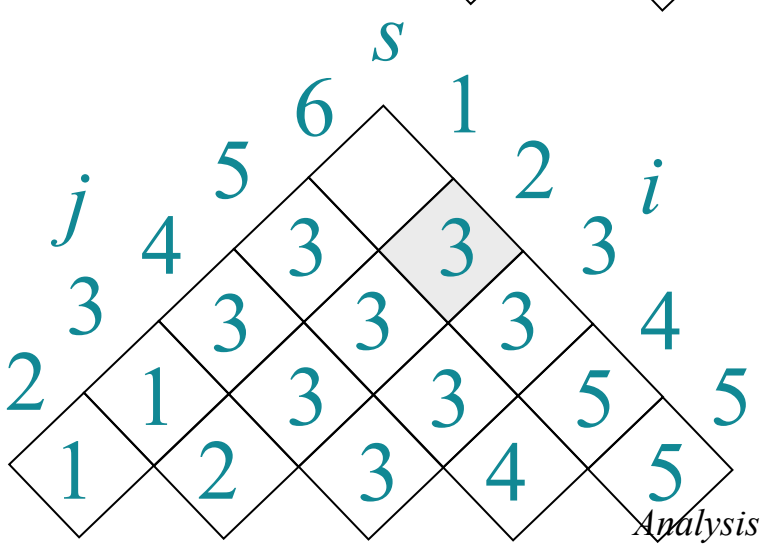
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25



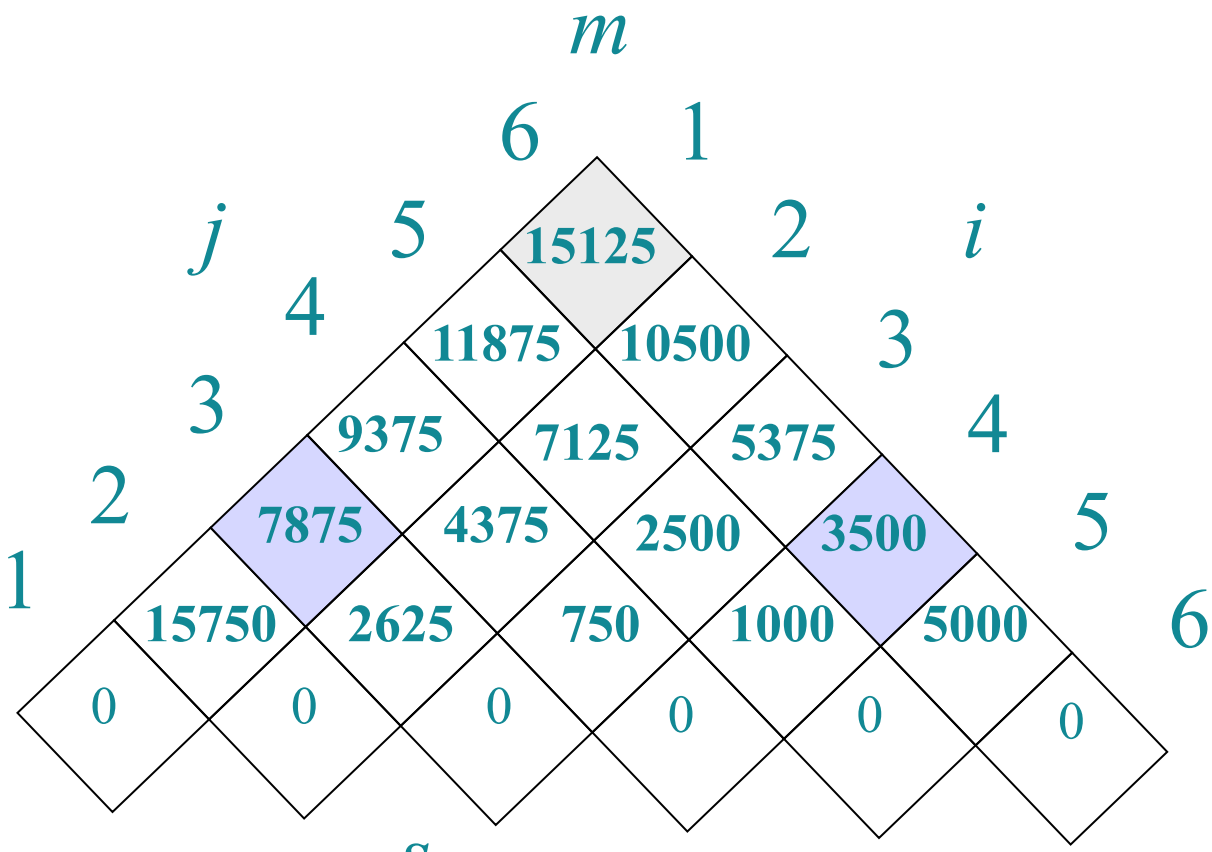


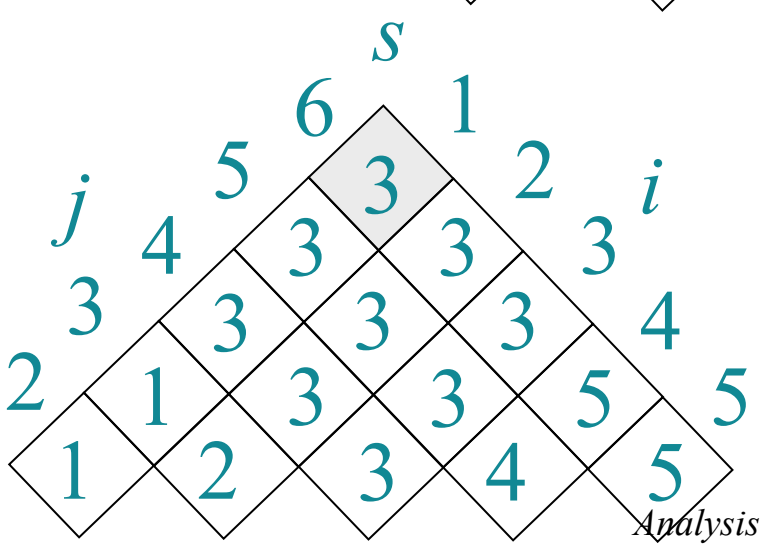
matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25





matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25





matrix	dimension
A1	30 x 35
A2	35 x 15
A3	15 x 5
A4	5 x 10
A5	10 x 20
A6	20 x 25

Print Parenthesization

PRINT-OPTIMAL-PARENS(s,i,j)

if $i = j$ **then**

print "A"$_i$

else print "("

PRINT-OPTIMAL-PARENS($s,i,s[i,j]$)

PRINT-OPTIMAL-PARENS($s,s[i,j]+1,j$)

print ")"

Recap (Matrix chain multiplication)

- Optimal substructure – leads to recursive solution
- Overlapping subproblems $\Theta(n^2)$ problems
- Dynamic programming – solve smaller subproblems before larger subproblems $\Theta(n^3)$